



자바응용SW(앱)개발자양성과정

애플리케이션 구성하기

백제직업전문학교

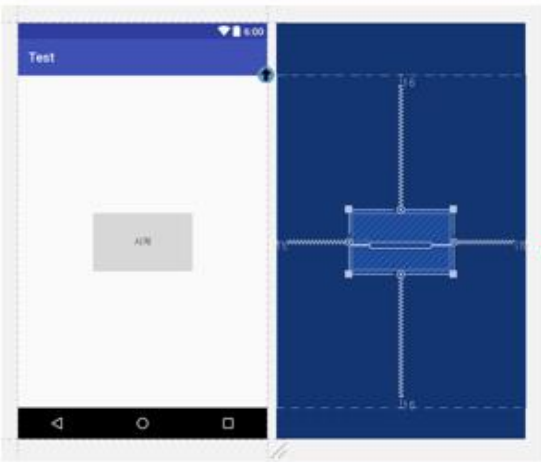
김영준 강사

1.

레이아웃 인플레이션

XML 레이아웃 파일과 자바 소스 파일의 매칭

- setContentView 메소드에서 XML 레이아웃 파일 매칭



XML 레이아웃 파일
activity_main.xml

```
...  
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
...  

```

자바 소스 코드 파일
MainActivity.java

실행

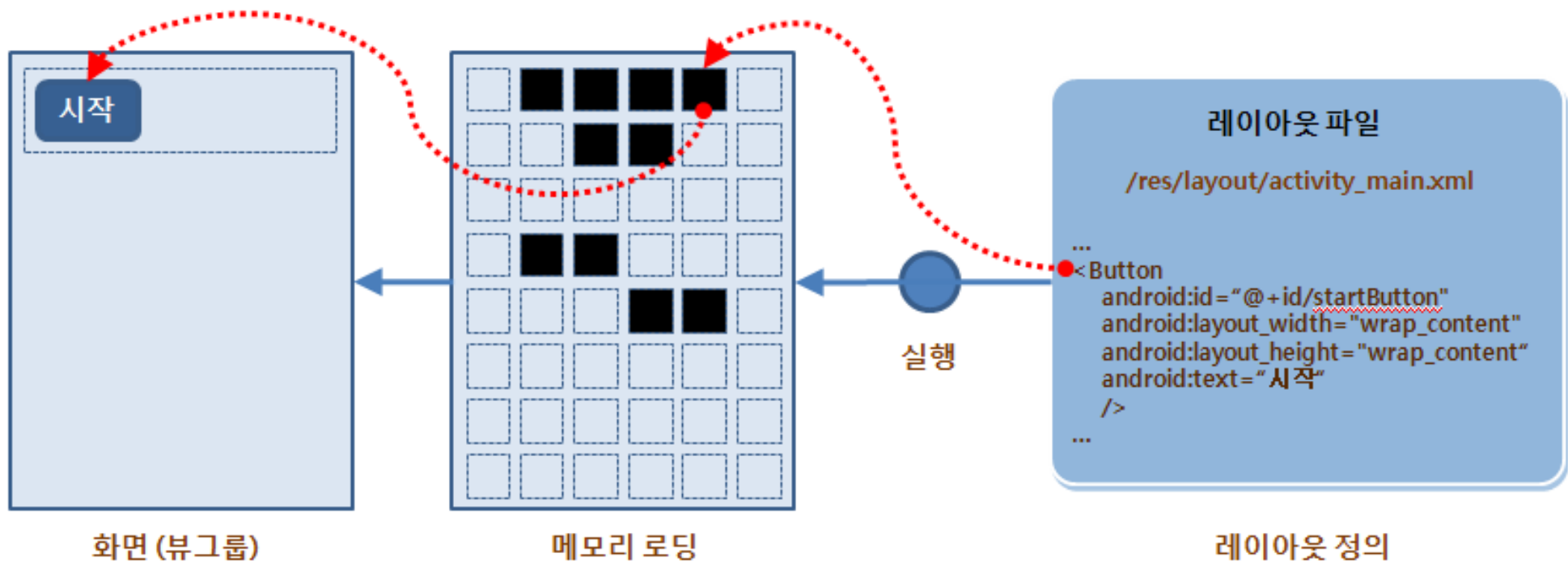


R.layout.레이아웃 파일 이름



인플레이션이란?

- 인플레이션 : XML 레이아웃에 정의된 내용이 메모리에 객체화되는 과정



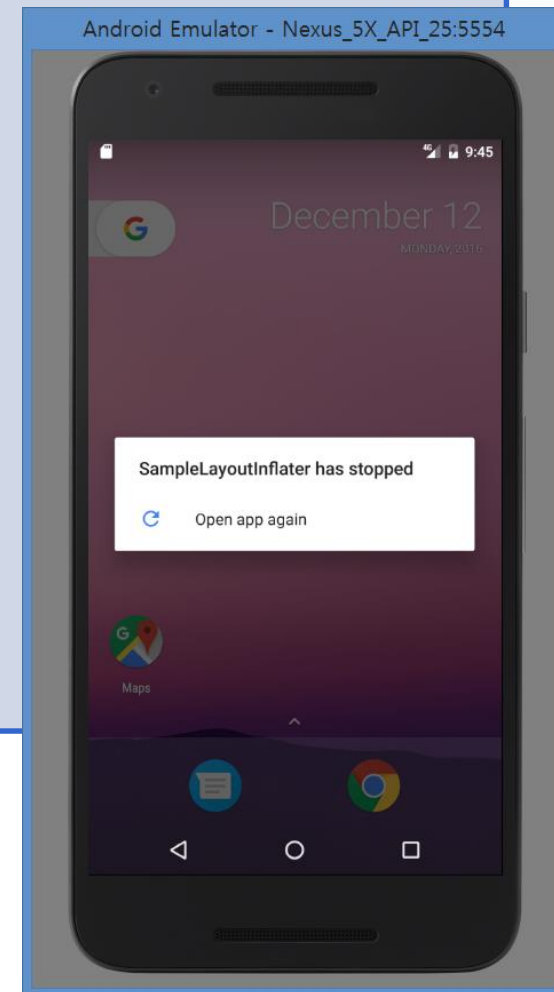
["시작" 버튼의 레이아웃 인플레이션 과정]



레이아웃 인플레이션의 이해 - 호출 순서

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
  
        Button button1 = (Button) findViewById(R.id.button1);  
        button1.setText( " 시작됨 " );  
  
        setContentView(R.layout.activity_main);  
    }  
}
```

[setContentView() 코드와 findViewById() 메소드의 호출 순서를 바꾼 경우]





setContentView() 메소드의 역할

[Reference]

```
public void setContentView (int layoutResID)
```

```
public void setContentView (View view [, ViewGroup.LayoutParams params])
```

- **setContentView() 메소드의 역할**

- 화면에 나타낼 뷰를 지정하는 역할
- XML 레이아웃의 내용을 메모리 상에 객체화하는 역할

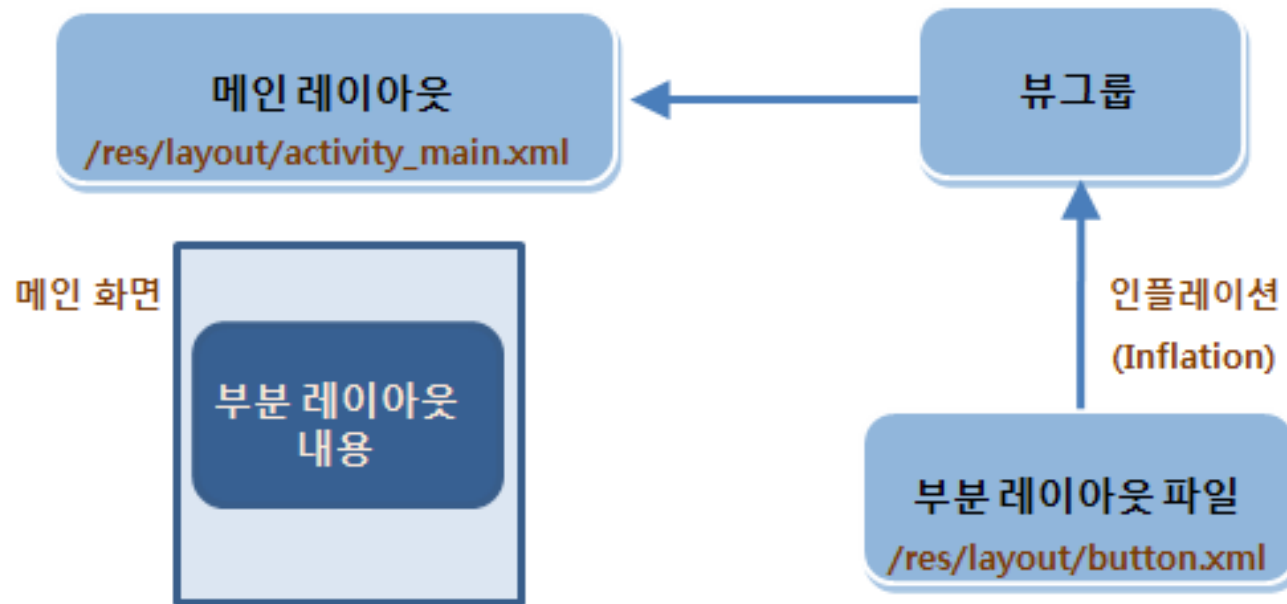
[Reference]

```
getSystemService(Context.LAYOUT_INFLATER_SERVICE)
```

- **전체 화면 중에서 일부분만을 차지하는 화면 구성요소들을 XML 레이아웃에서 로딩하여 보여 줄 수 있을까?**
 - LayoutInflater 라는 클래스를 제공하며, 이 클래스는 시스템 서비스로 제공됨



레이아웃 인플레이션의 개념도



[화면의 일부분을 XML 레이아웃 파일의 내용으로 적용하는 과정]



화면 전체와 화면 일부

- 안드로이드에서 화면 : 소스와 화면 구성이 분리되어 있음
 - 자바 소스 1개
 - XML 레이아웃 1개
- 화면 전체 : 액티비티 → setContentView 에서 인플레이션
 - 액티비티를 위한 자바 소스 1개 : MainActivity.java
 - 액티비티를 위한 XML 레이아웃 1개 : activity_main.xml
- 부분 화면 → 수동으로 인플레이션
 - 부분화면을 위한 자바 소스 1개 또는 뷰 (뷰가 1개의 소스 파일로 분리될 수 있음)
 - 부분화면을 위한 XML 레이아웃 1개 : singer.xml



레이아웃 인플레이션 예제

레이아웃 인플레이션 예제

- 화면의 일부로 추가할 뷰의 XML 레이아웃 정의
- 레이아웃 인플레이션 후 자바 코드에서 화면의 일부로 추가

메인 액티비티의
XML 레이아웃

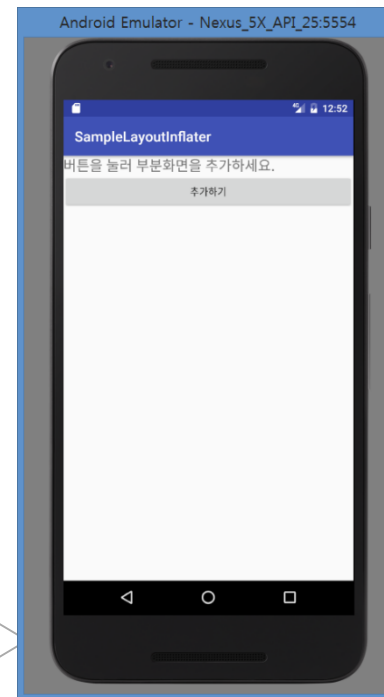
-레이아웃 코드 작성

화면 일부의
XML 레이아웃

-레이아웃 코드 작성

메인 액티비티 코드

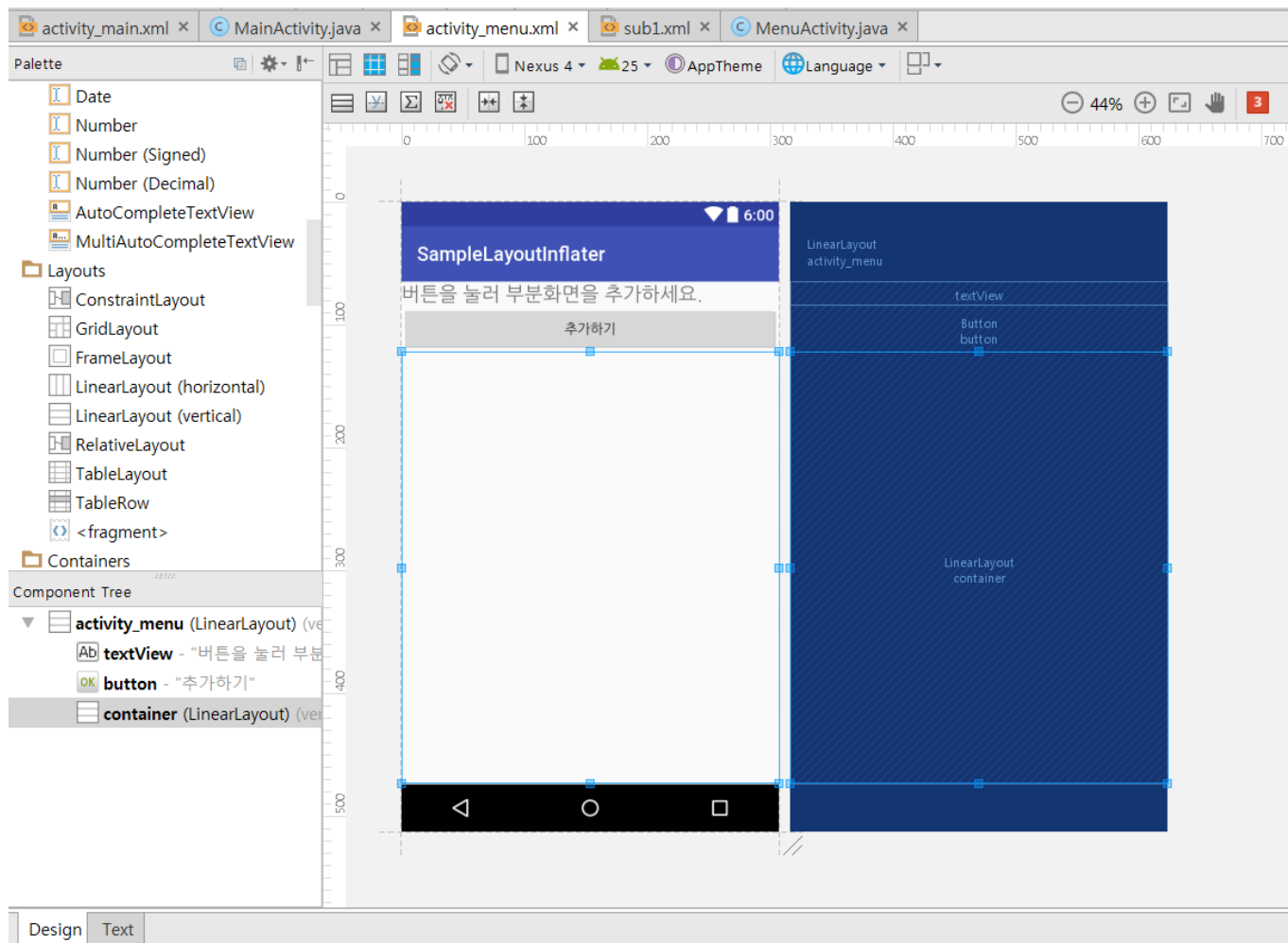
-메인 액티비티 코드 작성





메인 액티비티의 레이아웃

- 위쪽에 버튼을 배치하고 아래쪽에 다른 레이아웃이 들어갈 공간을 확보





메인 액티비티의 레이아웃

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/activity_menu"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

...

    <Button
        android:id="@+id/button2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="추가하기" />

    <LinearLayout
        android:id="@+id/container"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

    </LinearLayout>

</LinearLayout>
```



일부 화면을 위한 레이아웃

- sub1.xml 파일로 만들고 텍스트뷰와 체크박스 위젯 추가한 후 배경색 설정

The screenshot shows the Android Studio interface. On the left, the 'New Resource File' dialog is open, showing 'File name: sub1.xml', 'Root element: LinearLayout', and 'Source set: main'. The 'Available qualifiers' list includes 'Country Code', 'Network Code', 'Locale', 'Layout Direction', 'Smallest Screen Width', 'Screen Width', 'Screen Height', 'Size', and 'Ratio'. The 'Chosen qualifiers' section is empty. On the right, the 'Design' view of the layout is shown. The layout is a vertical LinearLayout with a blue background. It contains a TextView with the text '부분화면 1' and a CheckBox with the text '동의합니다.'. The 'Component Tree' on the left shows the hierarchy: LinearLayout (vertical) > TextView2 - '부분화면 1' > checkBox - '동의합니다.'.



자바 코드 작성

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_menu);  
  
    container = (LinearLayout) findViewById(R.id.container);  
  
    Button button = (Button) findViewById(R.id.button);  
    button.setOnClickListener(new View.OnClickListener() {  
        @Override  
        public void onClick(View view) {  
            LayoutInflater inflater = (LayoutInflater) getSystemService(Context.LAYOUT_INFLATER_SERVICE);  
            inflater.inflate(R.layout.sub1, container, true);  
  
            CheckBox checkBox = (CheckBox) container.findViewById(R.id.checkBox);  
            checkBox.setText("로딩되었습니다.");  
        }  
    });  
}
```

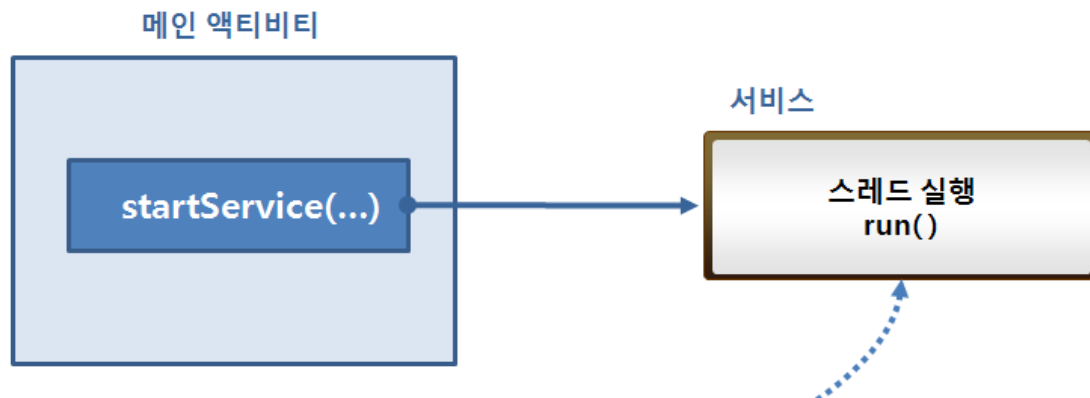


2.

서비스



서비스



- 서비스는 **백그라운드**에서 실행되는 애플리케이션 구성 요소
- 서비스는 매니페스트 파일(AndroidManifest.xml) 안에 **<service>** 태그를 이용하여 선언
- 서비스를 시작/중지시키는 메소드
 - Context.startService()
 - Context.bindService()
 - stopService(...)
 - unbindService(...)
- 서비스는 다른 구성 요소들처럼 메인 스레드에서 동작
따라서 CPU를 많이 쓰거나 대기 상태(blocking)를 필요로 하는 작업들은 **스레드를 새로** 만들어 주어야 함



서비스 실행 예제

서비스 예제

- 액티비티 상태에 따른 수명주기 확인하기
- 상태 메소드 별로 토스트 메시지 추가

메인 액티비티

startService(...)

서비스

스레드 실행
run()



매니페스트 파일
(AndroidManifest.xml)

서비스 클래스 정의

메인 액티비티 코드 작성

- 일정 시간간격으로 메시지를 보여 주는 서비스 클래스 정의
- 메인 액티비티에서 서비스 시작

매니페스트에 추가

- 새로운 서비스를 매니페스트에 추가



서비스 클래스 정의

```
package org.androidtown.basic.service;
import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.util.Log;

public class MyService extends Service implements Runnable {
    public static final String TAG = "MyService";
    private int count = 0;
    public void onCreate() {
        super.onCreate();
        Thread myThread = new Thread(this);
        myThread.start();
    }
```

1

시작

Continued..



서비스 클래스 정의 (계속)

```
public void run() {  
    while(true) {  
        try {  
            Log.i(TAG, "my service called #" + count);  
            count++;  
            Thread.sleep(5000);  
        } catch(Exception ex) {  
            Log.e(TAG, ex.toString());  
        }  
    }  
}  
  
@Override  
public IBinder onBind(Intent arg0) {  
    return null;  
}  
}
```

2

기록



메인 액티비티 코드 만들기

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        Intent myIntent = new Intent(this, MyService.class );  
        startService(myIntent);  
    }  
}
```

1

인텐트 객체 생성

2

서비스 시작



매니페스트에 추가하기

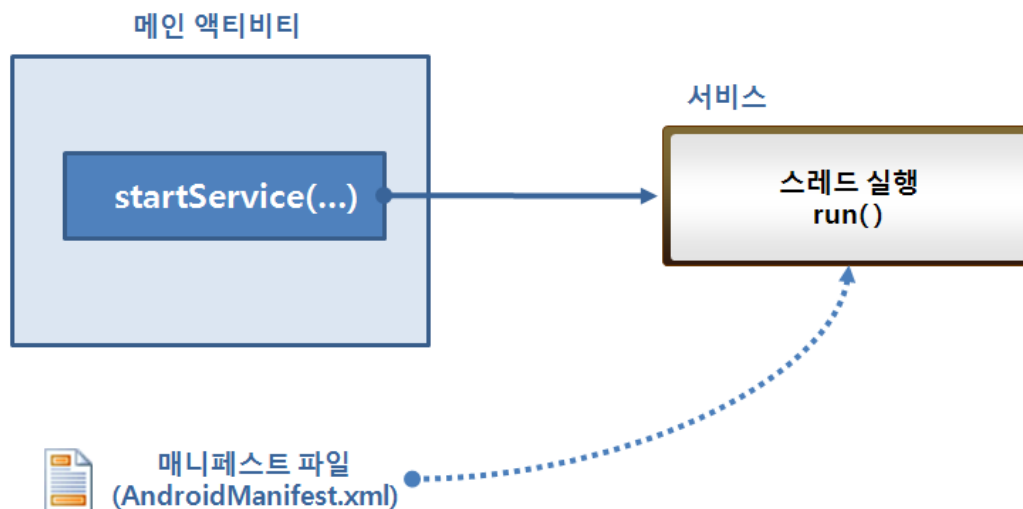
```
...  
<application android:icon="@drawable/icon" android:label="@string/app_name">  
...  
  <service android:name="MyService" > ← 1 서비스 등록  
  </service>  
  
</application>  
...
```



서비스 실행하여 로그 확인

Log				
Time	pid	tag	Message	
09-19 09:40:27.754	D 9746	ddm-heap	Got feature list request	
09-19 09:40:28.355	I 9746	MyService	my service called #0	
09-19 09:40:28.544	I 65	ActivityManager	Displayed activity org.androidtown	
09-19 09:40:33.364	I 9746	MyService	my service called #1	
09-19 09:40:33.754	D 243	dalvikvm	GC freed 6 objects / 176 bytes in	
09-19 09:40:38.367	I 9746	MyService	my service called #2	
09-19 09:40:38.815	D 144	dalvikvm	GC freed 2298 objects / 134760 byte	
09-19 09:40:43.364	I 9746	MyService	my service called #3	
09-19 09:40:48.372	I 9746	MyService	my service called #4	
09-19 09:40:53.374	I 9746	MyService	my service called #5	
09-19 09:40:58.377	I 9746	MyService	my service called #6	
09-19 09:41:03.379	I 9746	MyService		
09-19 09:41:08.382	I 9746	MyService		
09-19 09:41:13.384	I 9746	MyService		
09-19 09:41:18.385	I 9746	MyService		
09-19 09:41:23.387	I 9746	MyService		

[서비스를 통해 로그를 남긴 경우]



[서비스 이용 패턴]



서비스 예제

배경에서 음악을 연주하는 서비스를 작성하여 보자.





mp3 형식의 음악 파일을 하나 다운로드 받아서
/res/raw 디렉토리에 old_pop.mp3와 같은 이름으로
저장한다.



음악을 연주하는 서비스

```
public class MusicService extends Service {  
  
    private static final String TAG = "MusicService";  
    MediaPlayer player;  
  
    public IBinder onBind(Intent intent) {  
        return null;  
    }  
  
    public void onCreate() {  
        Log.d(TAG, "onCreate()");  
        player = MediaPlayer.create(this, R.raw.old_pop);  
        player.setLooping(false); // Set looping  
    }  
}
```




음악을 연주하는 서비스

```
public void onDestroy() {  
    Toast.makeText(this, "Music Service가 중지되었습니다.",  
        Toast.LENGTH_LONG).show();  
    Log.d(TAG, "onDestroy()");  
    player.stop();  
}  
public int onStartCommand(Intent intent, int flags, int startId) {  
    Toast.makeText(this, "Music Service가 시작되었습니다.",  
        Toast.LENGTH_LONG).show();  
    Log.d(TAG, "onStart()");  
    player.start();  
    return super.onStartCommand(intent, flags, startId);  
}  
}
```



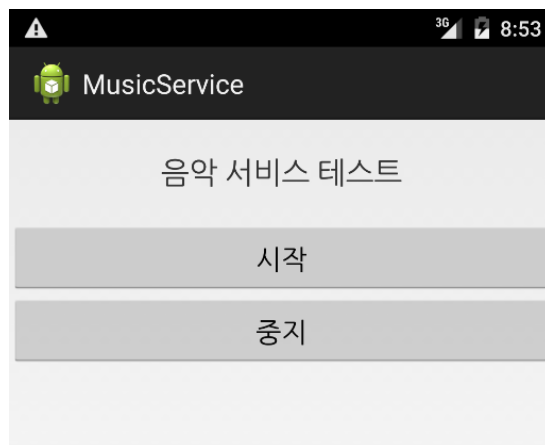
매니페스트 파일

```
<manifest
    ...
<application
    <activity          ...          </activity>
    <service
        android:enabled= "true"
        android:name= ".MusicService"
    />
</application>
</manifest>
```



서비스 사용 예제

앞의 서비스를 사용하는 예제를 작성
먼저 다음과 같은 인터페이스를 XML로 작성





사용자 인터페이스

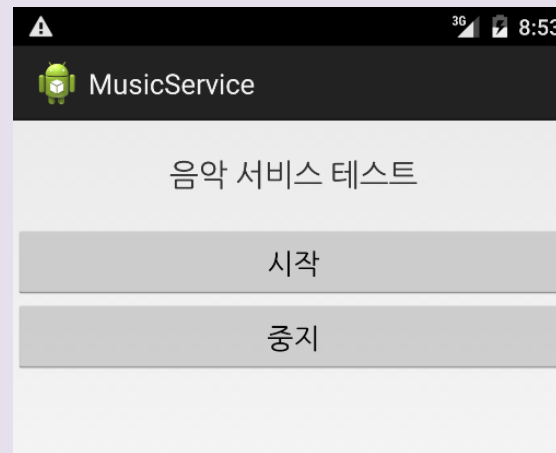
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="top/center"
    android:orientation="vertical" >

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:padding="20dp"
        android:text="음악 서비스 테스트"
        android:textSize="20sp" />

    <Button
        android:id="@+id/start"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="시작" >
    </Button>

    <Button
        android:id="@+id/stop"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="중지" >
    </Button>

</LinearLayout>
```





MusicServiceTest.java

...

```
public class MusicServiceTest extends Activity implements OnClickListener {
```

```
    private static final String TAG = "MusicServiceTest";
```

```
    Button start, stop;
```

```
    @Override
```

```
    public void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.main);
```

```
        start = (Button) findViewById(R.id.start);
```

```
        stop = (Button) findViewById(R.id.stop);
```

```
        start.setOnClickListener(this);
```

```
        stop.setOnClickListener(this);
```

```
}
```

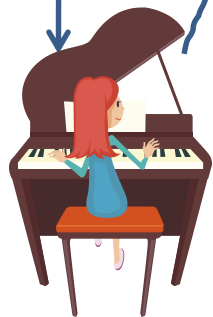
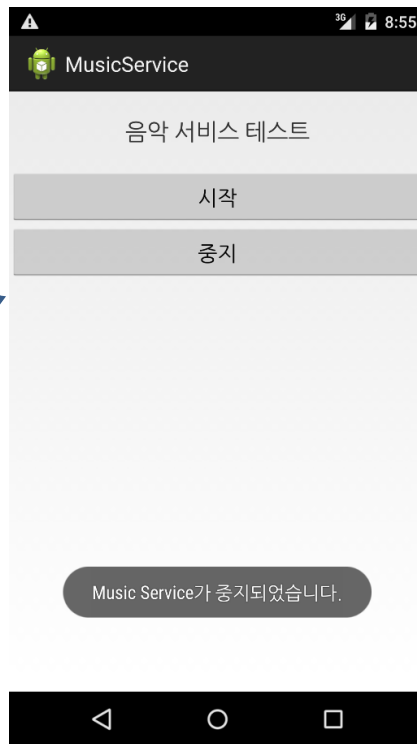
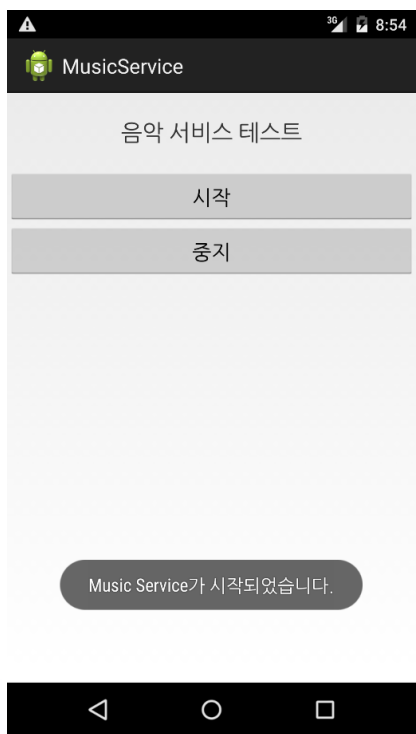


MusicServiceTest.java

```
public void onClick(View src) {  
    switch (src.getId()) {  
        case R.id.start:  
            Log.d(TAG, "onClick() start ");  
            startService(new Intent(this, MusicService.class));  
            break;  
  
        case R.id.stop:  
            Log.d(TAG, "onClick() stop");  
            stopService(new Intent(this, MusicService.class));  
            break;  
    }  
}  
}
```



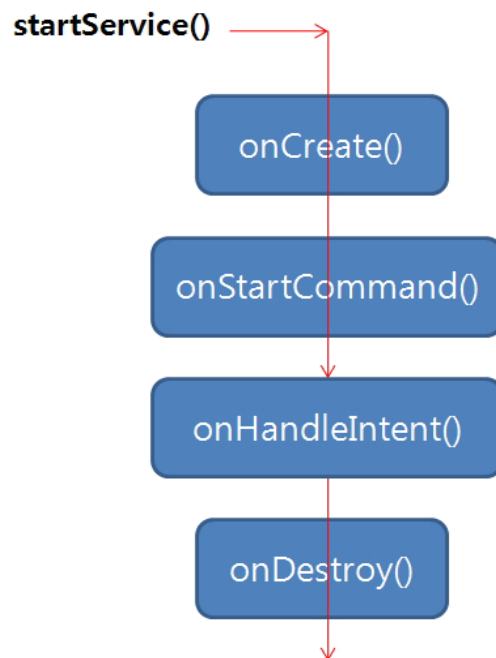
실행 결과





IntentService

- 인텐트 서비스는 자신에게 주어진 업무만 끝나면 자동으로 종료되는 특징





IntentService

```
public class MyIntentService extends IntentService {
    public MyIntentService() {
        super("MyIntentService");
    }
    public void onCreate() {
        super.onCreate();
        Log.d("kkang", "onCreate....");
    }
    public int onStartCommand(Intent intent, int flags, int startId) {
        Log.d("kkang", "onStartCommand....");
        return super.onStartCommand(intent, flags, startId);
    }
    public void onDestroy() {
        Log.d("kkang", "onDestroy....");
        super.onDestroy();
    }
    protected void onHandleIntent(Intent intent) {
        Log.d("kkang", "onHandleIntent start....");
        int sum=0;
        for(int i=1; i<11; i++){
            sum += i;
            SystemClock.sleep(1000);
        }
        Log.d("kkang", "onHandleIntent result:" + sum);
        Log.d("kkang", "onHandleIntent end....");
    }
}
```



bindService

- bindService () 함수로 실행

```
ServiceConnection connection=new ServiceConnection() {  
    @Override  
    public void onServiceConnected(ComponentName name, IBinder service) {  
    }  
    @Override  
    public void onServiceDisconnected(ComponentName name) {  
    }  
};
```

```
Intent bIntent=new Intent(this, BindService.class);  
bindService(bIntent, connection, Context.BIND_AUTO_CREATE);
```

- unbindService () 함수로 종료

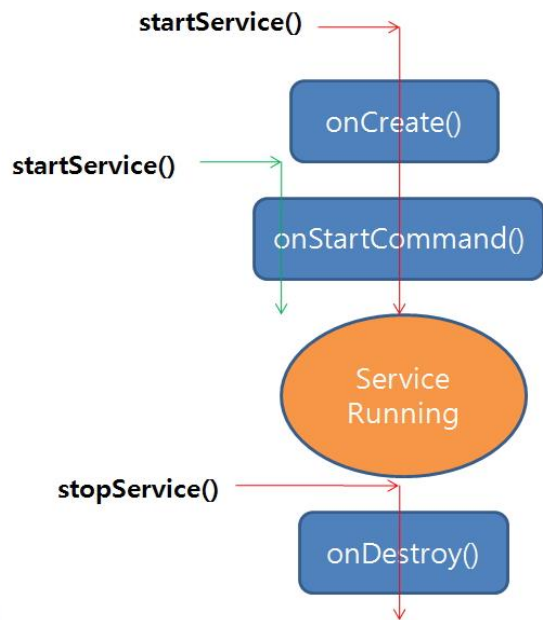
```
unbindService(connection);
```



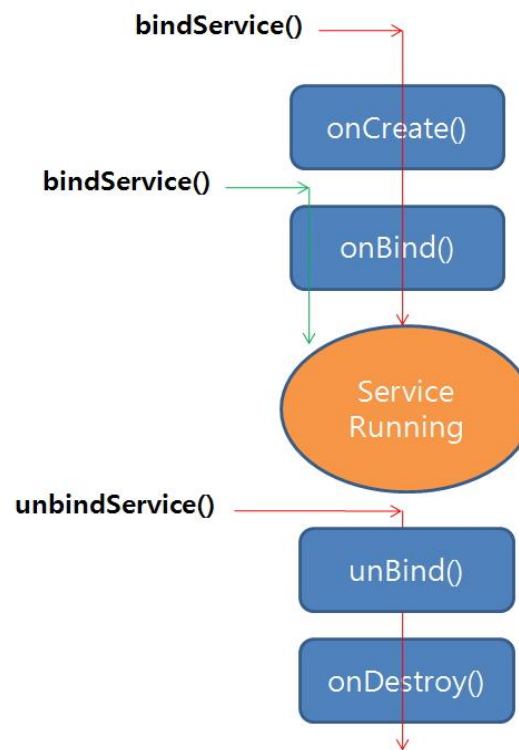
bindService

- 서비스 생명주기

- startService() 함수 이용 시 생명주기



- bindService() 함수 이용 시 생명주기





bindService

- onBind() 함수

```
public IBinder onBind(Intent intent) {  
    return new MyBinder();  
}
```

```
class MyBinder extends Binder{  
    public void some(){  
        //...  
    }  
}
```

```
ServiceConnection connection=new ServiceConnection() {  
    @Override  
    public void onServiceConnected(ComponentName name, IBinder service) {  
    }  
    @Override  
    public void onServiceDisconnected(ComponentName name) {  
    }  
};  
//.....  
Intent bIntent=new Intent(this, BindService.class);  
bindService(bIntent, connection, Context.BIND_AUTO_CREATE);
```

①
Service 실행

```
@Override  
public IBinder onBind(Intent intent) {  
    return new MyBinder();  
}
```

② 객체 리턴

```
class MyBinder extends Binder{  
    public void some(){  
        //.....  
    }  
}
```

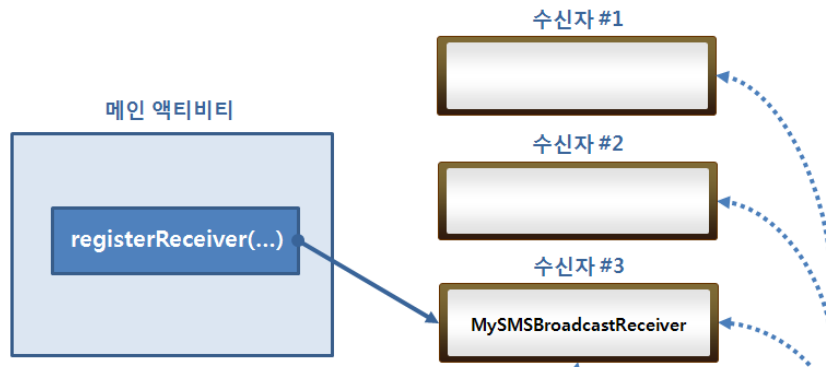


3.

브로드캐스트 수신자



브로드캐스트 수신자



- 애플리케이션이 글로벌 이벤트(global event)를 받아서 처리하려면 브로드캐스트 수신자로 등록
- 글로벌 이벤트란 "전화가 왔습니다.", "문자 메시지가 도착했습니다."와 같이 안드로이드 시스템 전체에 보내지는 이벤트
- 브로드캐스트 수신자는 인텐트필터를 포함하며, 매니페스트 파일에 등록함으로써 인텐트를 받을 준비를 함
- 수신자가 매니페스트 파일에 등록되었다면 따로 시작시키지 않아도 됨
- 애플리케이션은 컨텍스트 클래스의 registerReceiver 메소드를 이용하면 런타임 시에도 수신자를 등록할 수 있음
- 서비스처럼 브로드캐스트 수신자도 UI가 없음



브로드캐스트의 구분

• 인텐트와 브로드캐스트

- 인텐트를 이용해서 액티비티를 실행하면 포그라운드(background)로 실행되어 사용자에게 보여지지만
- 브로드캐스트를 이용해서 처리하면 백그라운드(background)로 동작하므로 사용자가 모름
- 인텐트를 받으면 onReceive() 메소드가 자동으로 호출됨

• 브로드캐스트의 구분

브로드캐스트는 크게 두 가지 클래스로 구분됨

- 일반 브로드캐스트 (sendBroadcast() 메소드로 호출)

비동기적으로 실행되며 모든 수신자는 순서없이 실행됨 (때로는 동시에 실행됨)

효율적이나, 한 수신자의 처리 결과를 다른 수신자가 이용할 수 없고 중간에 취소불가

- 순차 브로드캐스트 (sendOrderedBroadcast() 메소드로 호출)

한 번에 하나의 수신자에게만 전달되므로 순서대로 실행됨. 중간에 취소하면 그 다음

수신자는 받지 못함. 수신자가 실행되는 순서는 인텐트필터의 속성으로 정할 수 있음

순서가 같으면 임의로 실행됨.



브로드캐스트 수신자 예제

브로드캐스트 수신자 예제

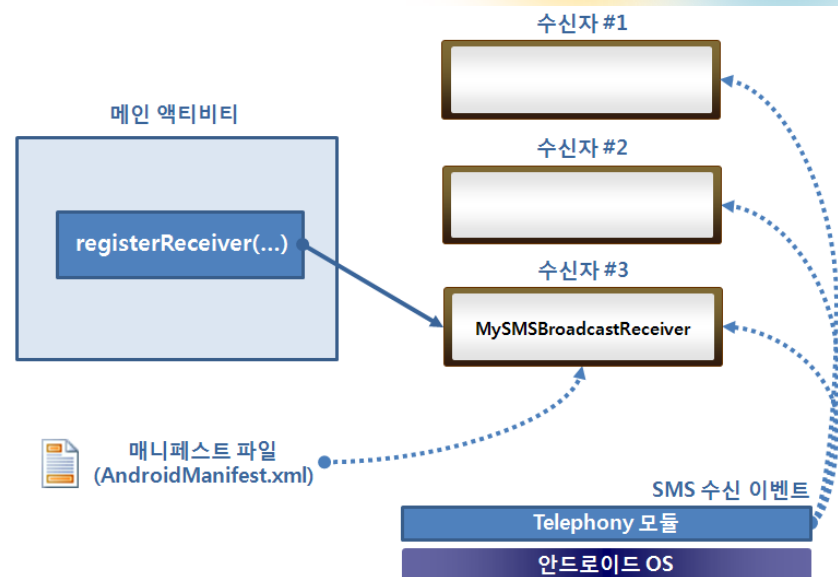
- 브로드캐스트 수신자로 SMS 수신 확인하기
- 브로드캐스트 수신자 정의

브로드캐스트 수신자 정의

- 일정 시간간격으로 메시지를 보여 주는 서비스 클래스 정의

매니페스트에 추가

- 새로운 브로드캐스트 수신자를 매니페스트에 추가





새로운 브로드캐스트 수신자 추가

- SampleReceiver 프로젝트 생성
- New → Other → Broadcast Receiver 메뉴로 브로드캐스트 수신자 추가

New Android Component

Configure Component
Android Studio

Creates a new broadcast receiver component and adds it to your Android manifest.

Class Name:

☒ Exported

☒ Enabled

Source Language:

Target Source Set:



매니페스트에 추가

```
<application
```

```
...
```

```
<receiver
```

```
  android:name=".SmsReceiver"
```

```
  android:enabled="true"
```

```
  android:exported="true">
```

```
    <intent-filter>
```

```
      <action android:name="android.provider.Telephony.SMS_RECEIVED"/>
```

```
    </intent-filter>
```

```
  </receiver>
```

```
</application>
```



브로드캐스트 수신자 클래스 정의

```
public class SmsReceiver extends BroadcastReceiver {  
    public static final String TAG = "SmsReceiver";  
  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        Log.i(TAG, "onReceive() 메소드 호출됨.");  
  
        Bundle bundle = intent.getExtras();  
        SmsMessage[] messages = parseSmsMessage(bundle);  
        if (messages != null && messages.length > 0) {  
            String sender = messages[0].getOriginatingAddress();  
            Log.i(TAG, "SMS sender : " + sender);  
            String contents = messages[0].getMessageBody().toString();  
            Log.i(TAG, "SMS contents : " + contents);  
            Date receivedDate = new Date(messages[0].getTimestampMillis());  
            Log.i(TAG, "SMS received date : " + receivedDate.toString());  
        }  
    }  
}
```

...



브로드캐스트 수신자 클래스 정의

```
...
private SmsMessage[] parseSmsMessage(Bundle bundle) {
    Object[] objs = (Object[]) bundle.get("pdus");
    SmsMessage[] messages = new SmsMessage[objs.length];

    int smsCount = objs.length;
    for (int i = 0; i < smsCount; i++) {
        // PDU 포맷으로 되어 있는 메시지를 복원합니다.
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) { // API 23 이상
            String format = bundle.getString("format");
            messages[i] = SmsMessage.createFromPdu((byte[]) objs[i], format);
        } else {
            messages[i] = SmsMessage.createFromPdu((byte[]) objs[i]);
        }
    }

    return messages;
}
...
```



권한 추가

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="org.androidtown.samplerceiver">

    <uses-permission android:name="android.permission.RECEIVE_SMS" />

    ...

```



Sync Now 버튼 클릭

```
activity_main.xml x MainActivity.java x SmsReceiver.java x AndroidManifest.xml x app x
Configure project in Project Structure dialog. Open Project Structure Hide notification
22 allprojects {
23     repositories {
24         maven { url 'https://jitpack.io' }
25     }
26 }
27
28 dependencies {
29     implementation fileTree(dir: 'libs', include: ['*.jar'])
30     implementation 'androidx.appcompat:appcompat:1.1.0'
31     implementation 'androidx.constraintlayout:constraintlayout:1.1.3'
32     testImplementation 'junit:junit:4.12'
33     androidTestImplementation 'androidx.test.ext:junit:1.1.1'
34     androidTestImplementation 'androidx.test.espresso:espresso-core:3.2.0'
35
36     implementation 'com.github.pedroSG94:AutoPermissions:1.0.3'
37 }
38
```



위험 권한 부여 코드 추가

참조파일 SampleReceiver>/app/java/org.techtown.receiver/MainActivity.java

```
public class MainActivity extends AppCompatActivity
    implements AutoPermissionsListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        AutoPermissions.Companion.loadAllPermissions(this, 101);
    }

    @Override public void onRequestPermissionsResult(int requestCode, String permissions[],
        int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        AutoPermissions.Companion.parsePermissions(this, requestCode, permissions, this);
    }

    @Override
    public void onDenied(int requestCode, @NotNull String[] permissions) {
        Toast.makeText(this, "permissions denied : " + permissions.length,
            Toast.LENGTH_LONG).show();
    }

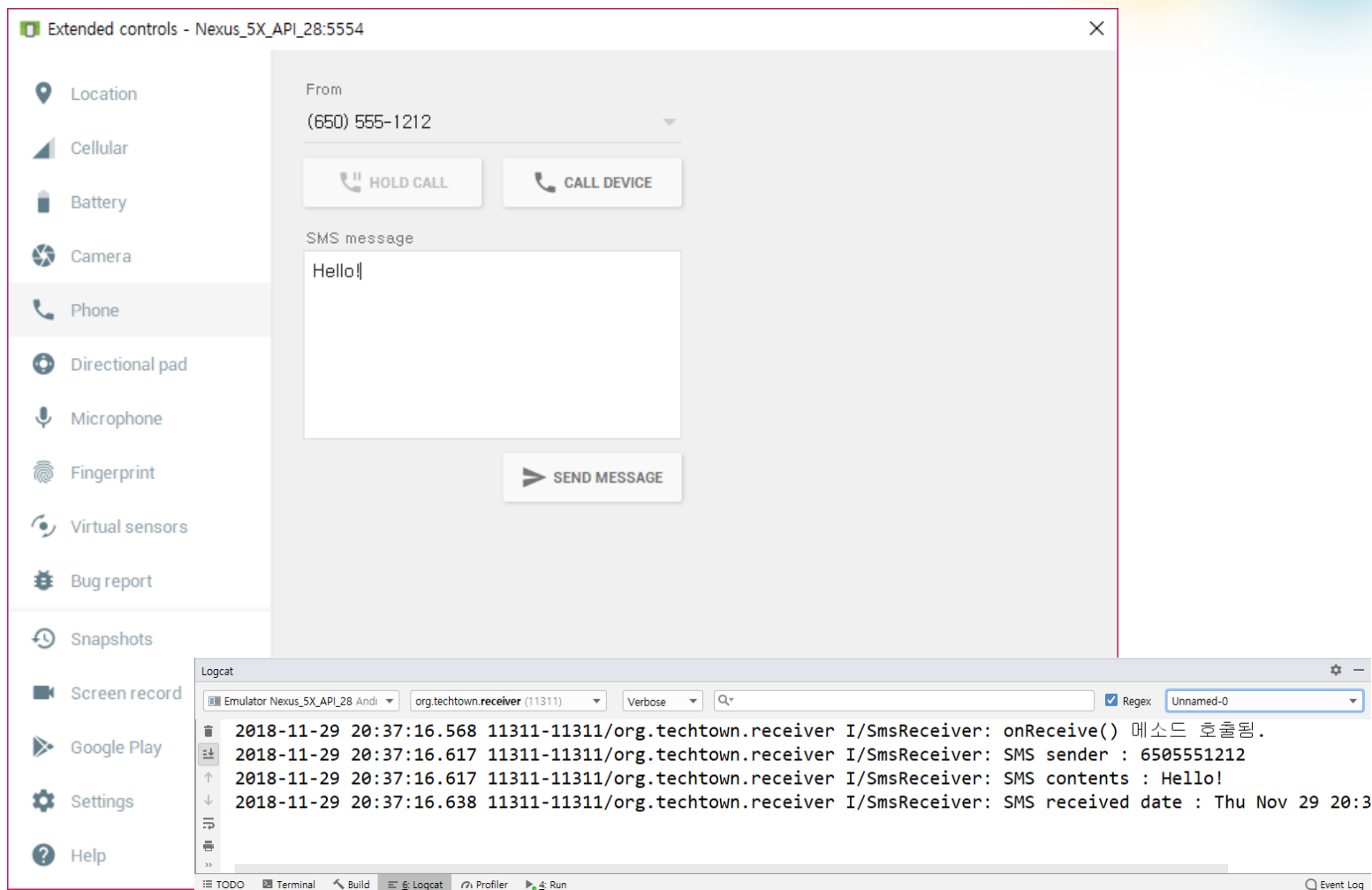
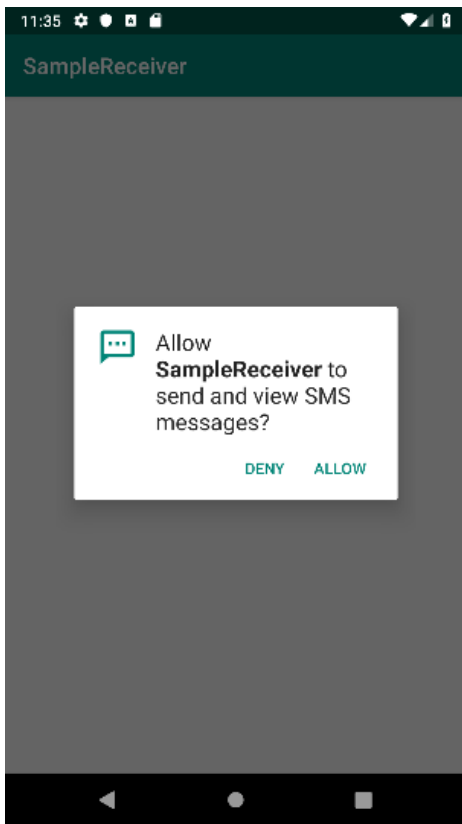
    @Override public void onGranted(int requestCode, @NotNull String[] permissions) {
        Toast.makeText(this, "permissions granted : " + permissions.length,
            Toast.LENGTH_LONG).show();
    }
}
```

① MainActivity가 인터페이스 구현하도록 하기

→ ② 모든 위험 권한을 자동 부여하도록 하는 메서드 호출하기



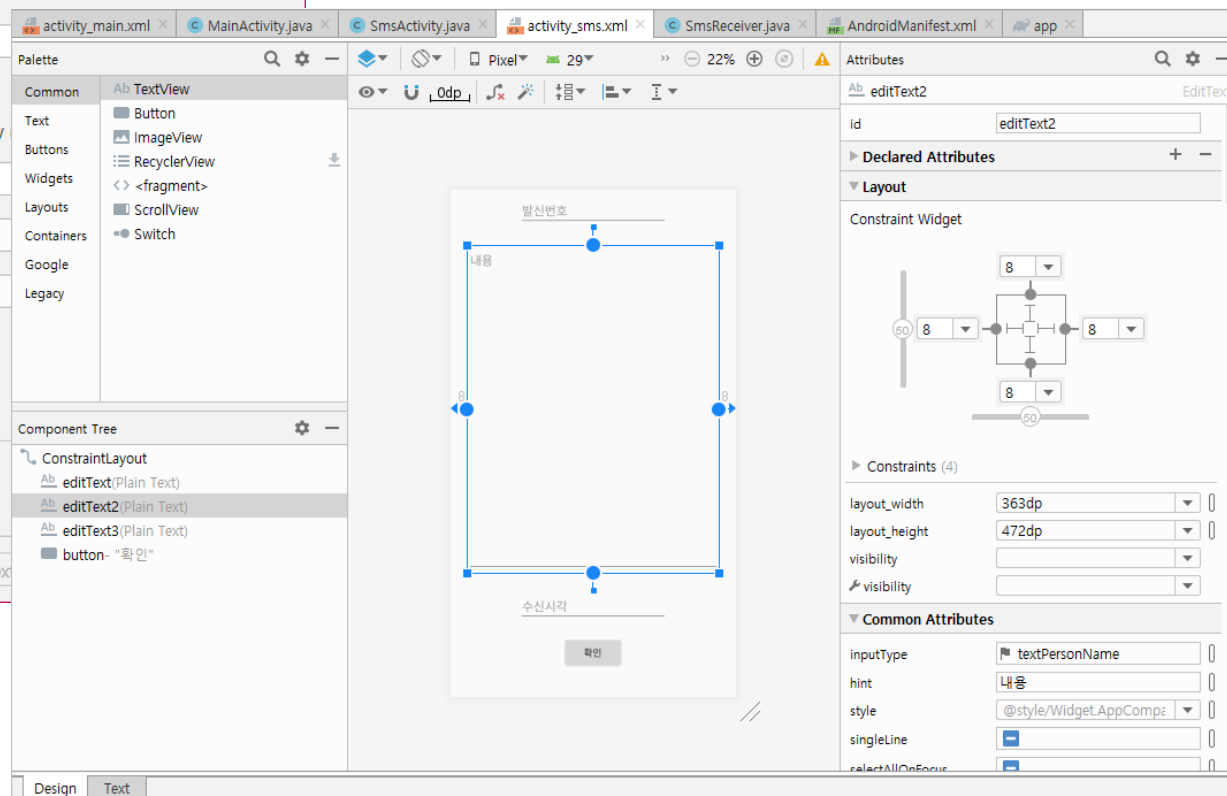
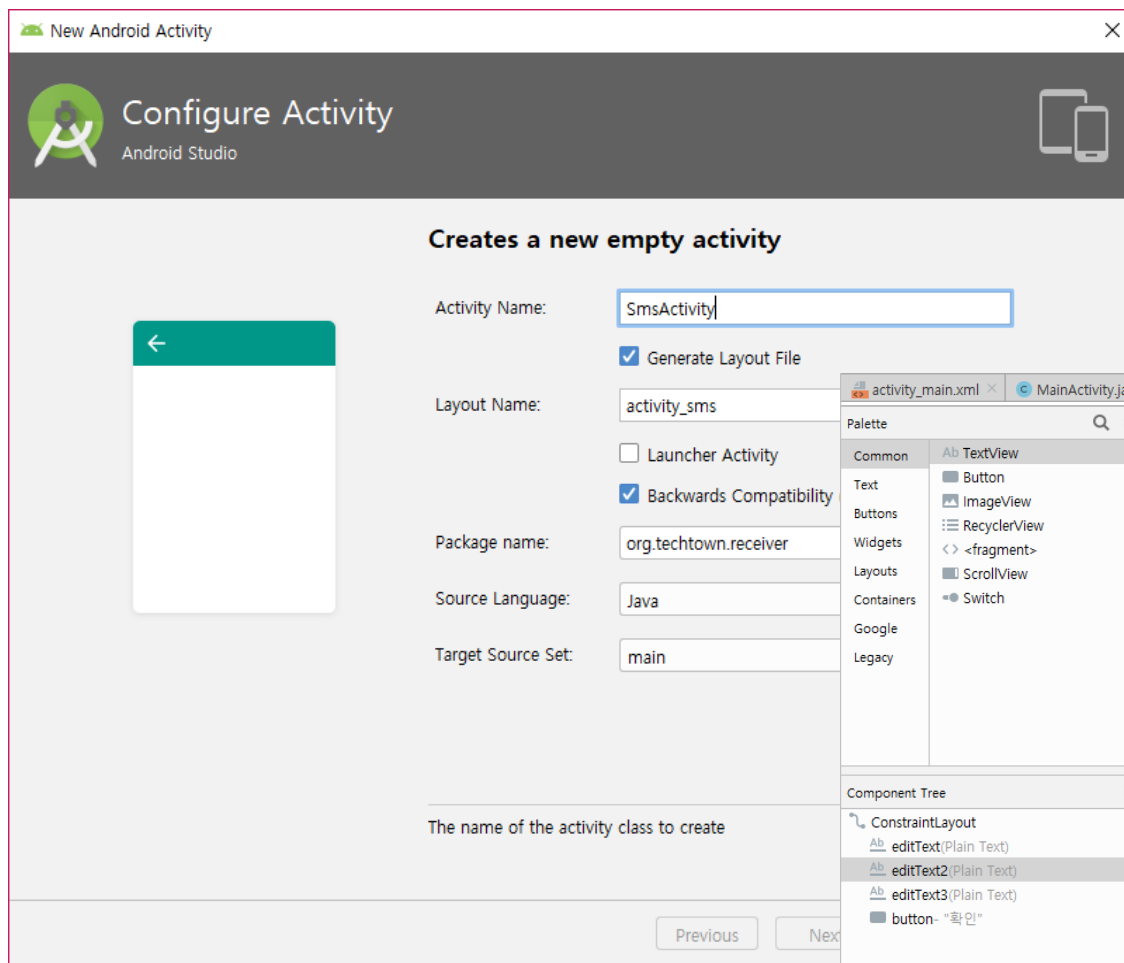
브로드캐스트 수신자 실행 화면



[Extended Controls 에서 에뮬레이터로 SMS를 보내고 로그가 출력된 화면]



SMS 문자를 보여줄 새로운 화면 추가



2. 브로드캐스트 수신자 이해하기



SMS 문자를 보여줄 새로운 화면 추가

```
...
@Override
protected void onNewIntent(Intent intent) {
    processIntent(intent);

    super.onNewIntent(intent);
}

private void processIntent(Intent intent) {
    if (intent != null) {
        String sender = intent.getStringExtra("sender");
        String contents = intent.getStringExtra("contents");
        String receivedDate = intent.getStringExtra("receivedDate");

        editText.setText(sender);
        editText2.setText(contents);
        editText3.setText(receivedDate);
    }
}
...
```



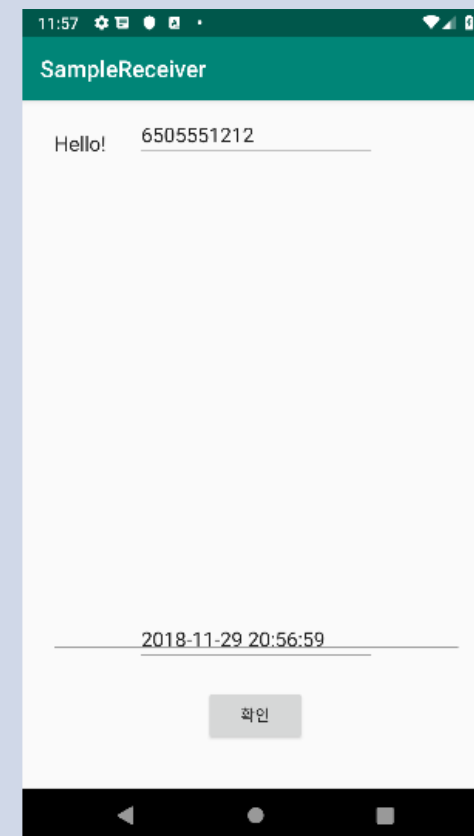
서비스에서 화면 띄우기

```
...
private void sendToActivity(Context context, String sender, String contents, Date receivedDate) {
    // 메시지를 보여줄 액티비티를 띄워줍니다.
    Intent myIntent = new Intent(context, SmsActivity.class);

    // 플래그를 이용합니다.
    myIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK|
        Intent.FLAG_ACTIVITY_SINGLE_TOP|Intent.FLAG_ACTIVITY_CLEAR_TOP);

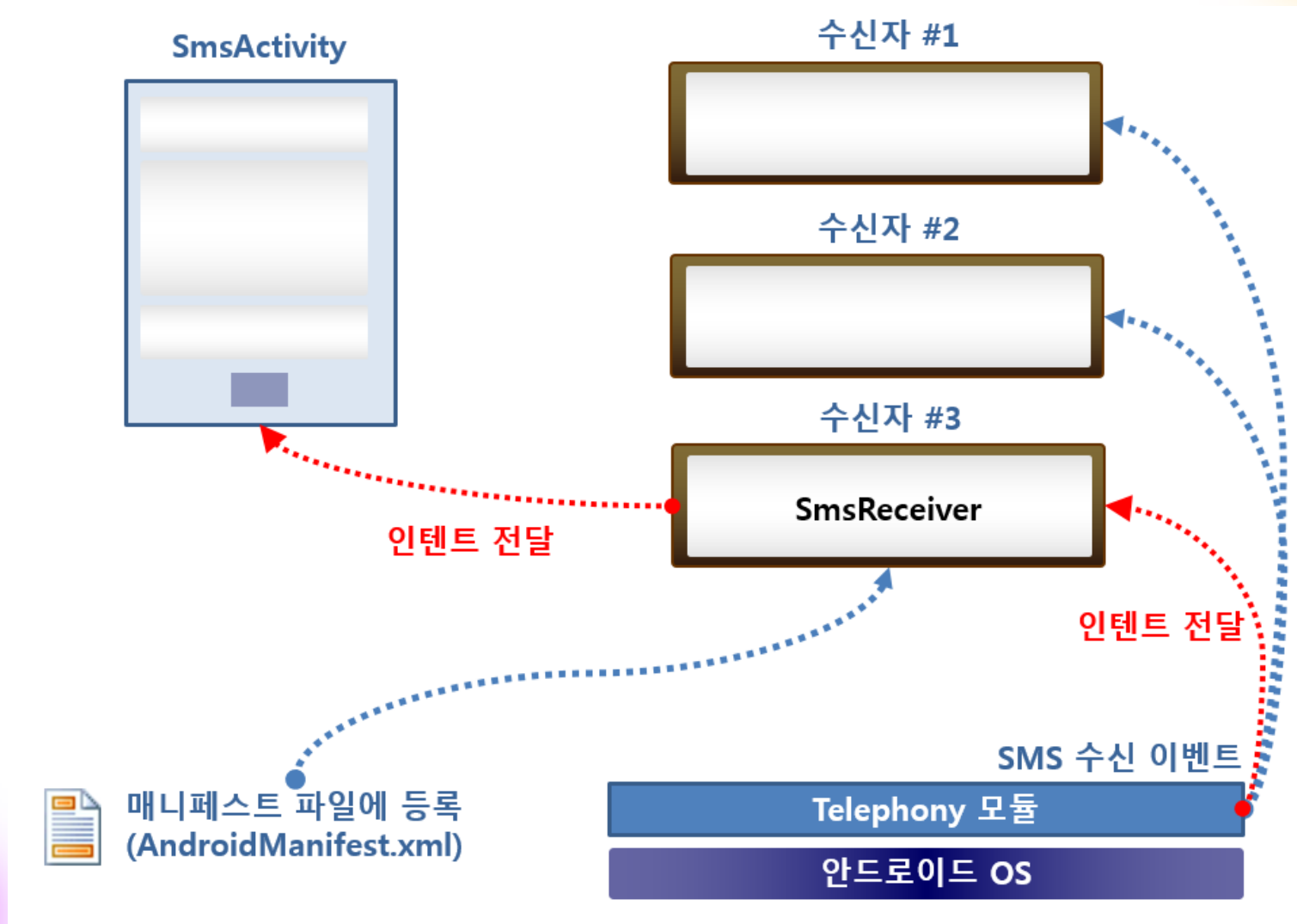
    myIntent.putExtra("sender", sender);
    myIntent.putExtra("contents", contents);
    myIntent.putExtra("receivedDate", format.format(receivedDate));

    context.startActivity(myIntent);
}
...
```





브로드캐스트 수신자 동작 방식

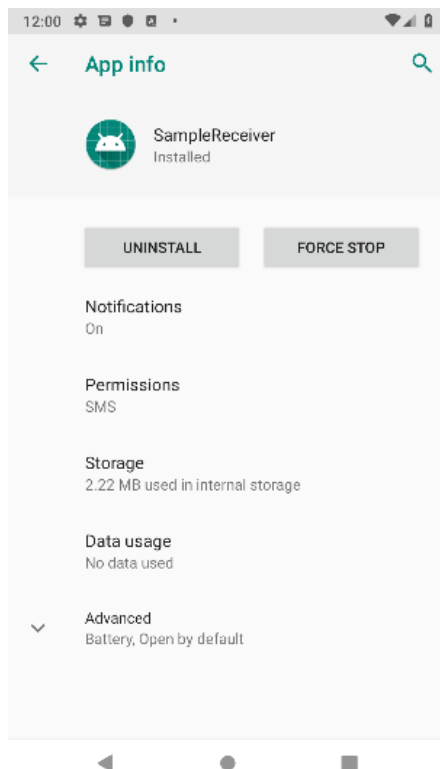




화면을 수정해도 이전 화면이 계속 보여요!

- 프로젝트를 복사하여 새로운 프로젝트를 만든 경우

- SMS를 수신하는 동일한 앱이 설치되어 있다면 SMS 수신 시 그 화면이 보여질 수 있음
- 따라서, 이전에 설치한 앱을 삭제한 후 새로운 앱을 실행해야 함
- 단말의 설정에서 앱을 삭제할 수 있음





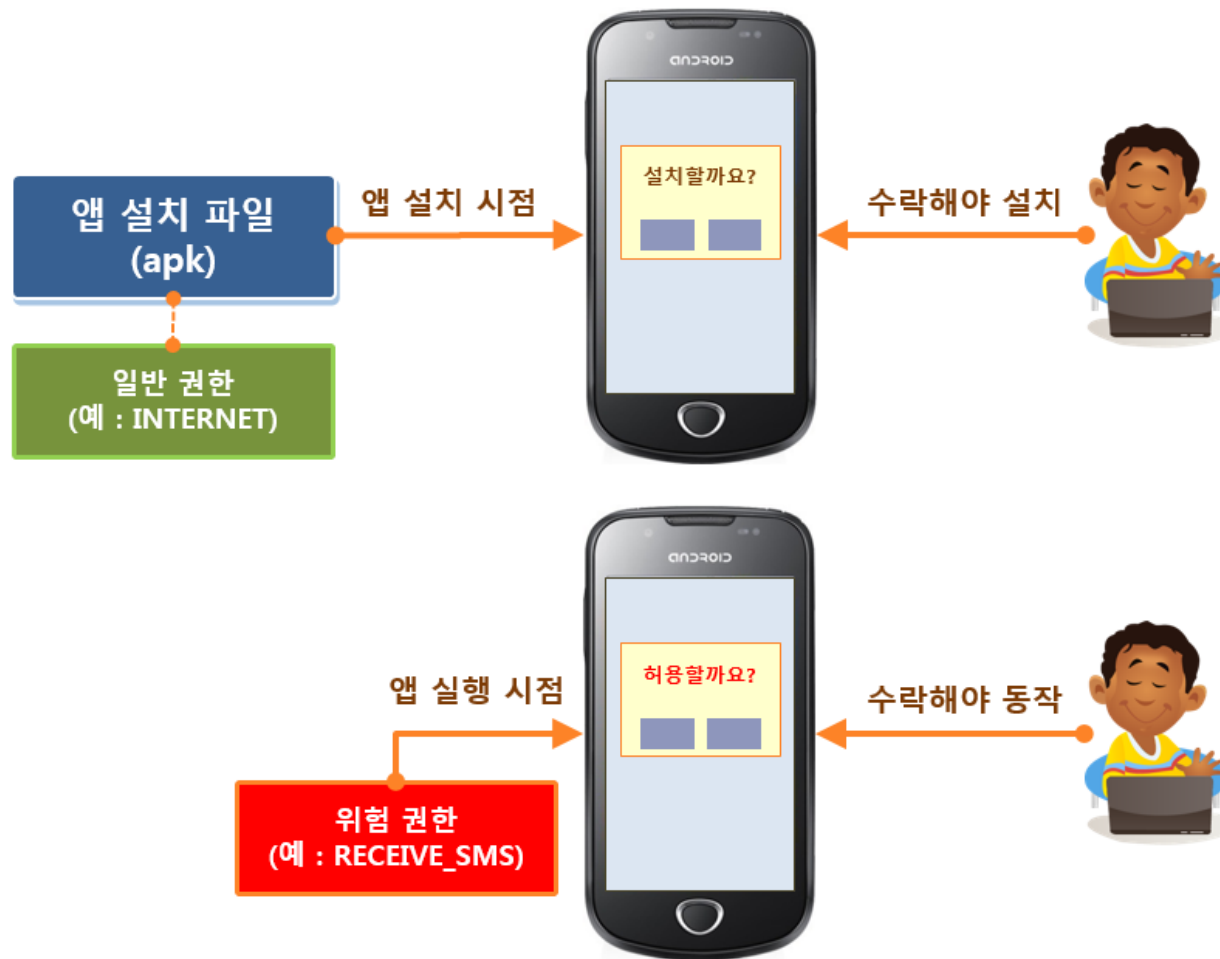
4.

위험 권한 부여하기



일반 권한과 위험 권한 (마시멜로 API23부터)

- 위험 권한은 실행 시 권한 부여





대표적인 위험 권한들



LOCATION (위치)

- ACCESS_FINE_LOCATION
- ACCESS_COARSE_LOCATION

CAMERA

- CAMERA

MICROPHONE

- RECORD_AUDIO

CONTACTS

- READ_CONTACTS
- WRITE_CONTACTS
- GET_ACCOUNTS

PHONE

- READ_PHONE_STATE
- CALL_PHONE
- READ_CALL_LOG
- WRITE_CALL_LOG
- ADD_VOICEMAIL
- USE_SIP
- PROCESS_OUTGOING_CALLS

SMS

- SEND_SMS
- RECEIVE_SMS
- READ_SMS
- RECEIVE_WAP_PUSH
- RECEIVE_MMS

CALENDAR

- READ_CALENDAR
- WRITE_CALENDAR

SENSORS

- BODY_SENSORS

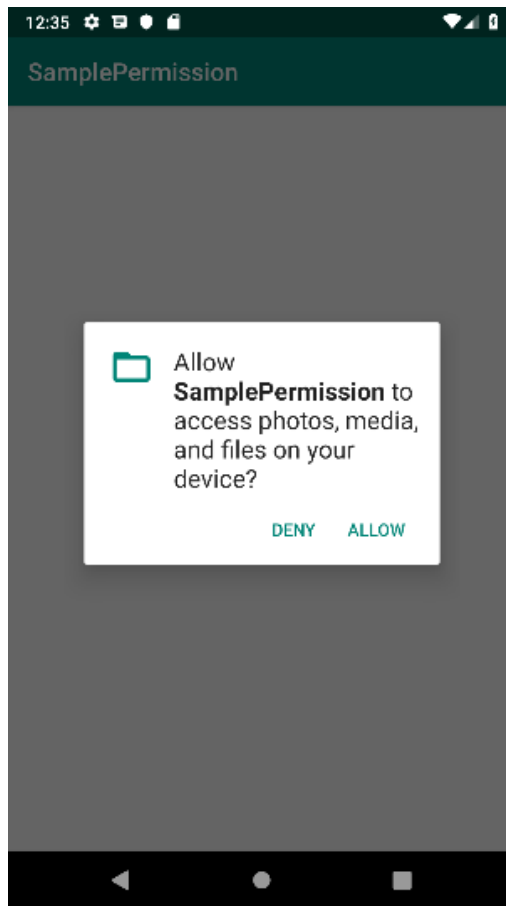
STORAGE

- READ_EXTERNAL_STORAGE
- WRITE_EXTERNAL_STORAGE



실행 시 권한 부여

- 실행 시 권한 부여를 묻는 대화상자 표시





매니페스트에 권한 추가

참조파일 SamplePermission>/app/manifests/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="org.techtown.permission">

    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

    <application
```

중략...



위험 권한 부여 요청 코드 추가

```
public void checkPermissions(String[] permissions) {
    ArrayList<String> targetList = new ArrayList<String>();

    for (int i = 0; i < permissions.length; i++) {
        String curPermission = permissions[i];
        int permissionCheck = ContextCompat.checkSelfPermission(this, curPermission);
        if (permissionCheck == PackageManager.PERMISSION_GRANTED) {
            Toast.makeText(this, curPermission + " 권한 있음.", Toast.LENGTH_LONG).show();
        } else {
            Toast.makeText(this, curPermission + " 권한 없음.", Toast.LENGTH_LONG).show();
            if (ActivityCompat.shouldShowRequestPermissionRationale(this, curPermission)) {
                Toast.makeText(this, curPermission + " 권한 설명 필요함.",
                    Toast.LENGTH_LONG).show();
            } else {
                targetList.add(curPermission);
            }
        }
    }

    String[] targets = new String[targetList.size()];
    targetList.toArray(targets);

    ActivityCompat.requestPermissions(this, targets, 101); —→ ❷ 위험 권한 부여 요청하기
```



권한 요청 결과 확인 코드

```
@Override
public void onRequestPermissionsResult(int requestCode, String permissions[],
                                      int[] grantResults) {
    switch (requestCode) {
        case 1: {
            if (grantResults.length > 0 &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                Toast.makeText(this, "SMS 권한을 사용자가 승인함.",
                              Toast.LENGTH_LONG).show();
            } else {
                Toast.makeText(this, "SMS 권한 거부됨.", Toast.LENGTH_LONG).show();
            }

            return;
        }
    }
}
```

Handwritten annotations: A red circle with '1' points to the `switch (requestCode)` line. A red circle with '2' points to the `return;` line.



위험 권한 자동 부여

참조파일 SamplePermission2>Gradle Scripts>build.gradle(Module:app)

중략...

```
allprojects {  
    repositories {  
        maven { url 'https://jitpack.io' }  
    }  
}
```

```
dependencies {
```

중략...

```
    implementation 'com.github.pedroSG94:AutoPermissions:1.0.3'  
}
```



위험 권한 자동 부여 코드 추가

참조파일 SamplePermission2>/app/java/org.techtown.permission2/MainActivity.java

```
public class MainActivity extends AppCompatActivity
    implements AutoPermissionsListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        AutoPermissions.Companion.loadAllPermissions(this, 101); → ❶
    }

    @Override
    public void onRequestPermissionsResult(int requestCode, String permissions[],
                                           int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        AutoPermissions.Companion.parsePermissions(this, requestCode, permissions, this);
    }

    @Override
    public void onDenied(int requestCode, @NotNull String[] permissions) {
        Toast.makeText(this, "permissions denied : " + permissions.length,
            Toast.LENGTH_LONG).show();
    }

    @Override
    public void onGranted(int requestCode, @NotNull String[] permissions) {
        Toast.makeText(this, "permissions granted : " + permissions.length,
            Toast.LENGTH_LONG).show();
    }
}
```





5.

프래그먼트



프래그먼트란?

- **프래그먼트 (Fragment)**

- 화면의 일정 영역을 독립적으로 처리하기 위해 만들어진 특별한 화면 구성 요소
- 태블릿의 대화면에서 화면 분할이 필요하게 되면서 만들어짐

- **프래그먼트의 기본 목적**

- 하나의 화면이 XML 레이아웃과 자바 소스로 구성된다는 점에 착안하여 하나의 프래그먼트가 XML 레이아웃과 자바 소스로 구성되도록 하고 독립적으로 관리되도록 하기 위함

- **기본 구성 방식**

- 액티비티가 동작하는 방식을 본떠 만들었음

- **사용되는 개념**

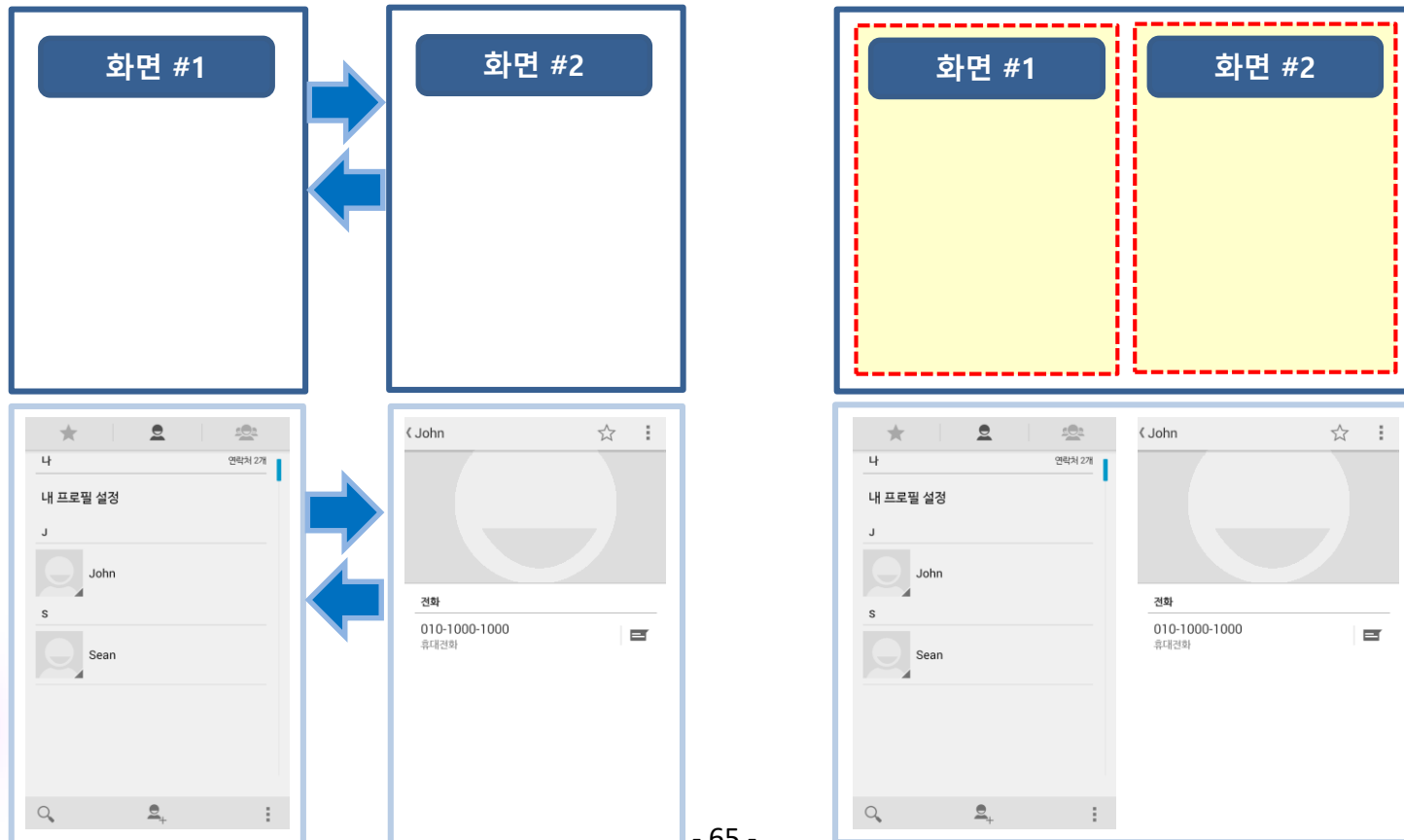
- 프래그먼트 매니저 : 프래그먼트를 관리하는 객체
- 트랜잭션 : 프래그먼트의 처리를 위해 만든 단위



화면을 전환하는 경우와 화면을 분할하는 경우

• 화면 전환과 화면 분할

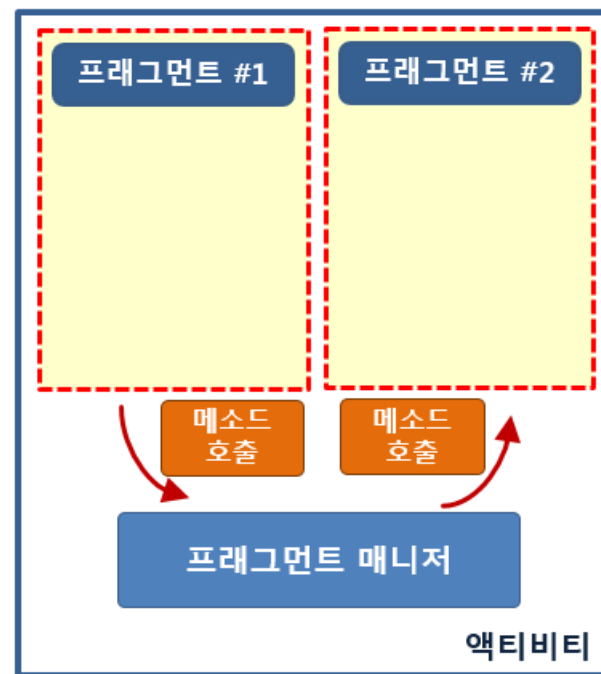
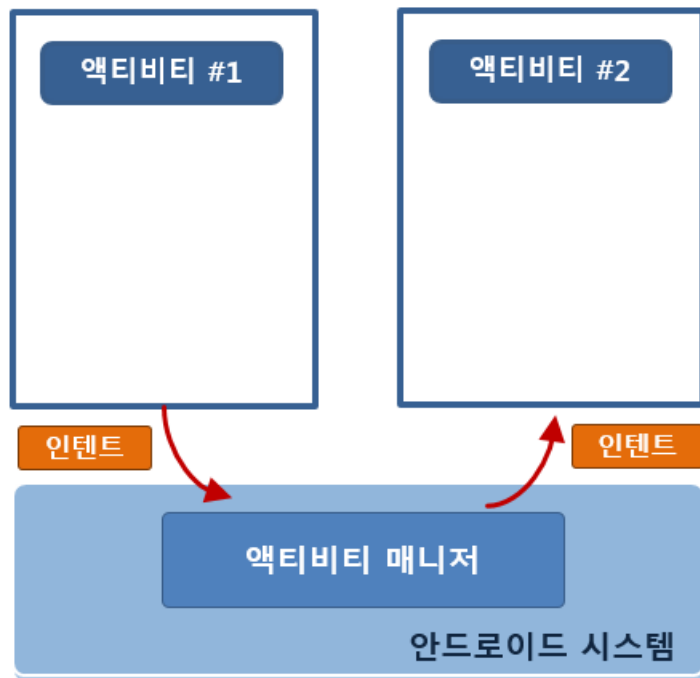
- (1) 두 개의 화면을 액티비티로 만들고 액티비티 간 전환
- (2) 하나의 액티비티 위에 프래그먼트를 두고 프래그먼트 간 전환
- ✓ 프래그먼트로 만들어 두면 스마트폰에서는 프래그먼트 간 화면 전환, 태블릿에서는 두 개의 프래그먼트를 하나의 액티비티 위에서 동시에 보여주는 화면 분할이 가능함





액티비티와 프래그먼트의 동작 방식 비교

- 액티비티를 시스템의 액티비티 매니저에서 관리하듯이 프래그먼트를 액티비티의 프래그먼트 매니저에서 관리함
 - 프래그먼트 입장에서는 액티비티가 시스템 역할을 하므로 프래그먼트는 항상 액티비티 위에 올라가 있어야 함

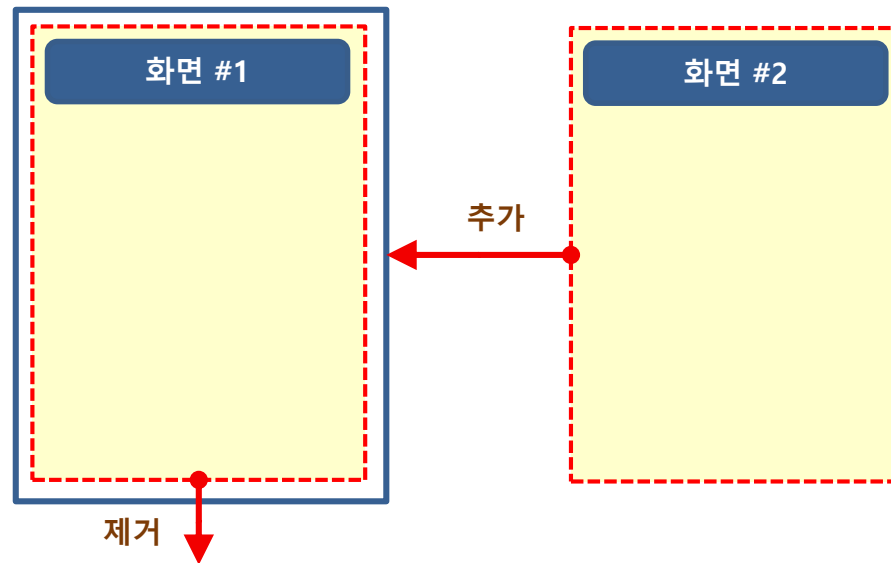




하나의 액티비티 안에서 프래그먼트 전환

• 프래그먼트 전환

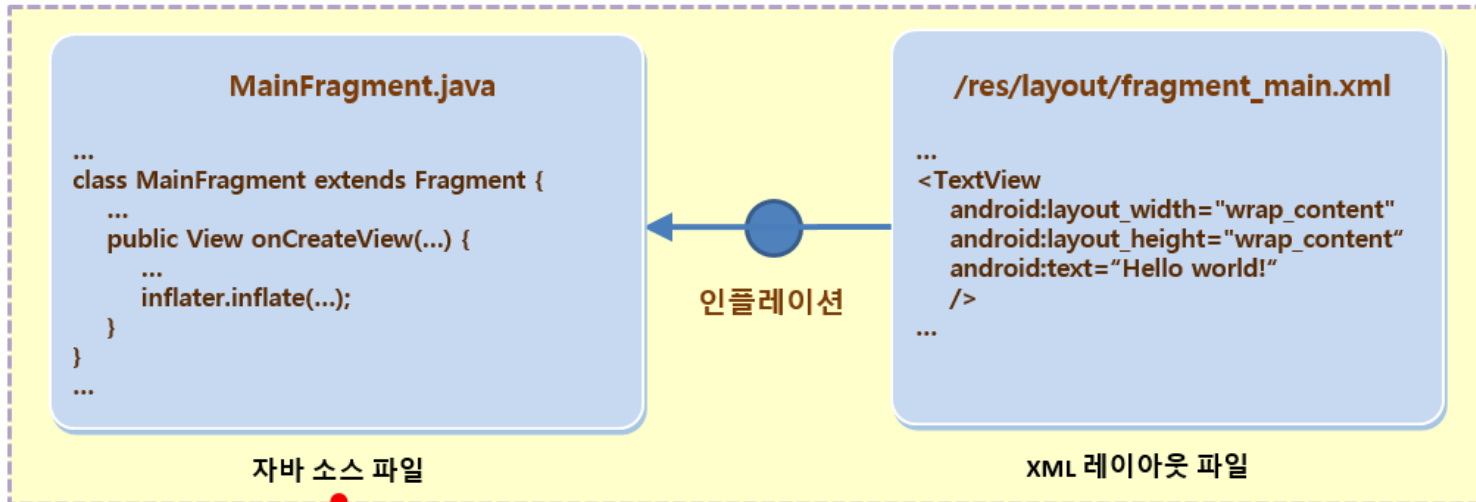
- 프래그먼트 매니저와 트랜잭션을 이용해 추가(add)나 교체(replace) 가능
- 싱글 프래그먼트라고 부르며 액티비티 전환 없이 화면 전체가 전환되는 효과를 낼 수 있음





프래그먼트를 위한 레이아웃 인플레이션

하나의 프래그먼트



```
<fragment  
    android:id="@+id/fragment"  
    android:name="org.androidtown.fragment.MainFragment"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" />
```



프래그먼트 만들어 화면에 추가하기

• 프래그먼트 추가

Creates a blank fragment that is compatible back to API level 4.

Fragment Name:

☒ Create layout XML?

Fragment Layout Name:

☒ Include fragment factory methods?

☒ Include interface callbacks?

Source Language:

Target Source Set:

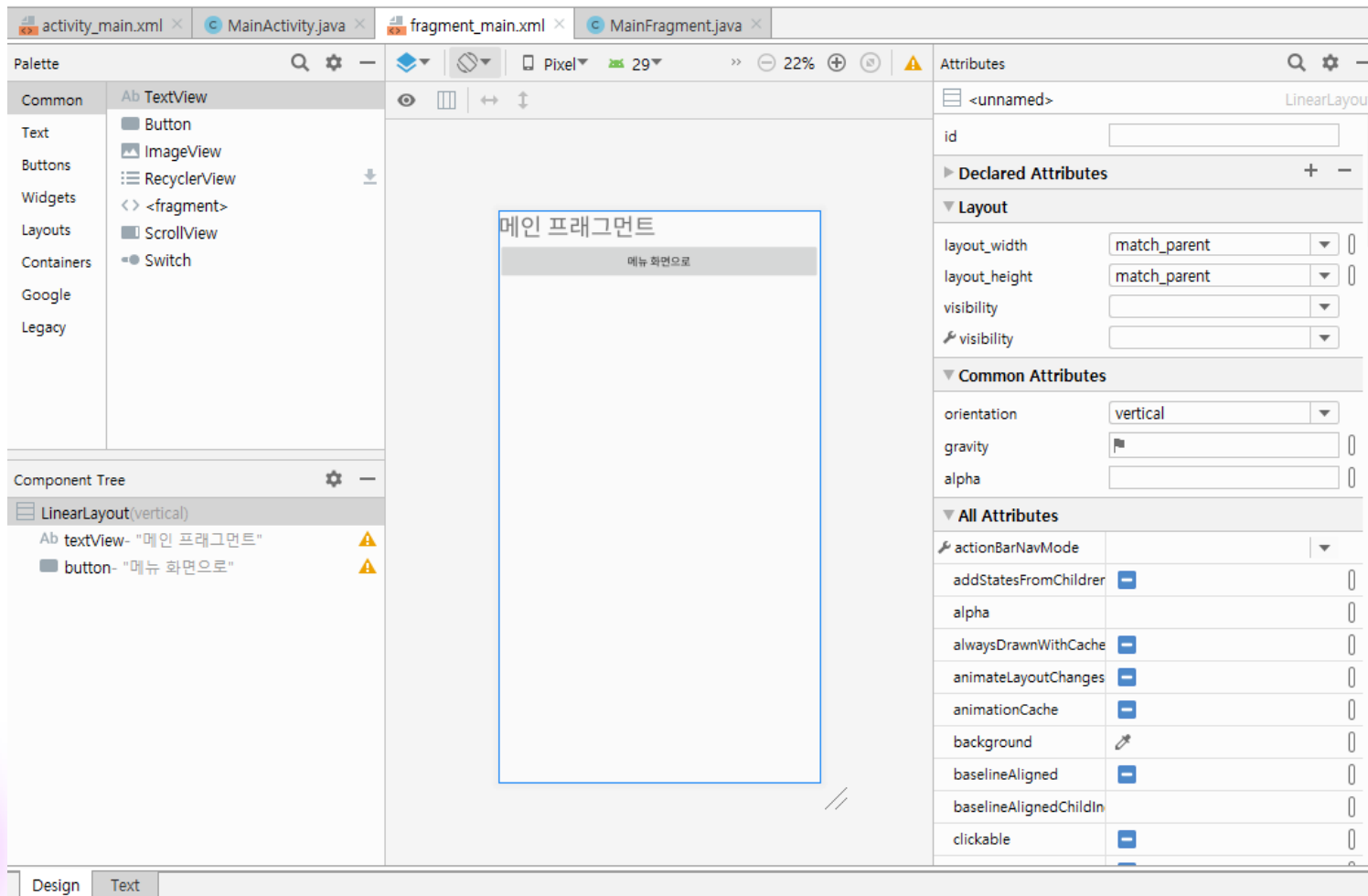
The name of the fragment class to create

Previous Next Cancel Finish



프래그먼트 만들어 화면에 추가하기

- 프래그먼트를 위한 화면 구성





프래그먼트 만들어 화면에 추가하기

- MainFragment.java 파일 만들고 프래그먼트 상속

참조파일 SampleFragment>/app/java/org.techtown.fragment/MainFragment.java

```
public class MainFragment extends Fragment {  
  
    @Override  
    public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {  
        return inflater.inflate(R.layout.fragment_main, container, false);  
    }  
}
```



프래그먼트 만들어 화면에 추가하기

- 메인 액티비티의 XML 레이아웃에 추가

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/container"
    >

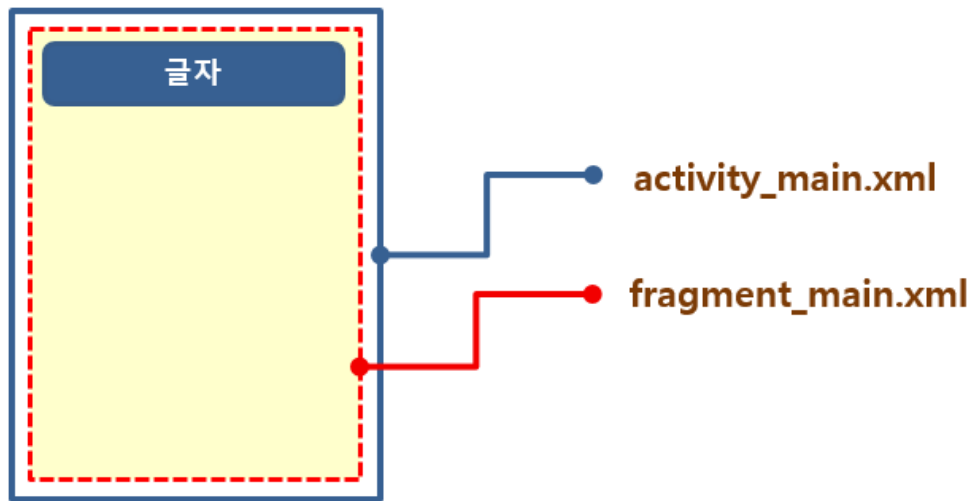
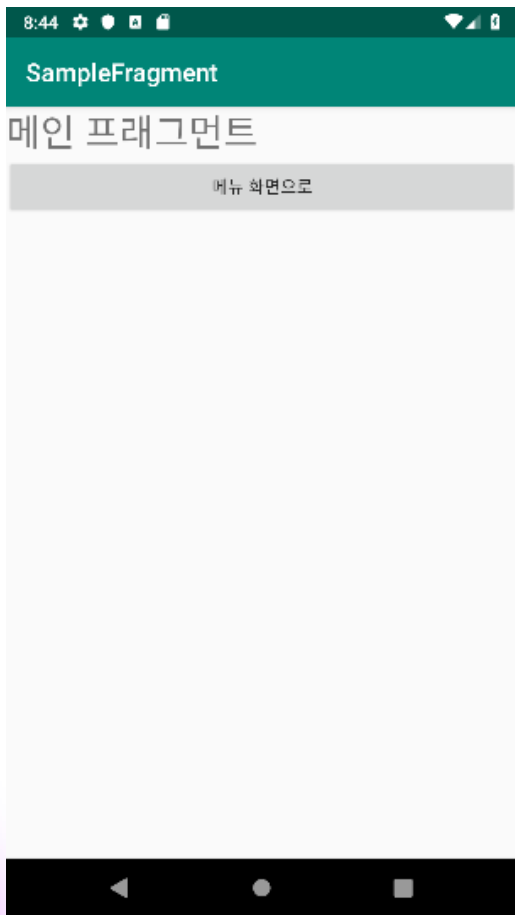
    <fragment
        android:id="@+id/mainFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:name="org.techtown.fragment.MainFragment" />

</RelativeLayout>
```




앱 실행 결과

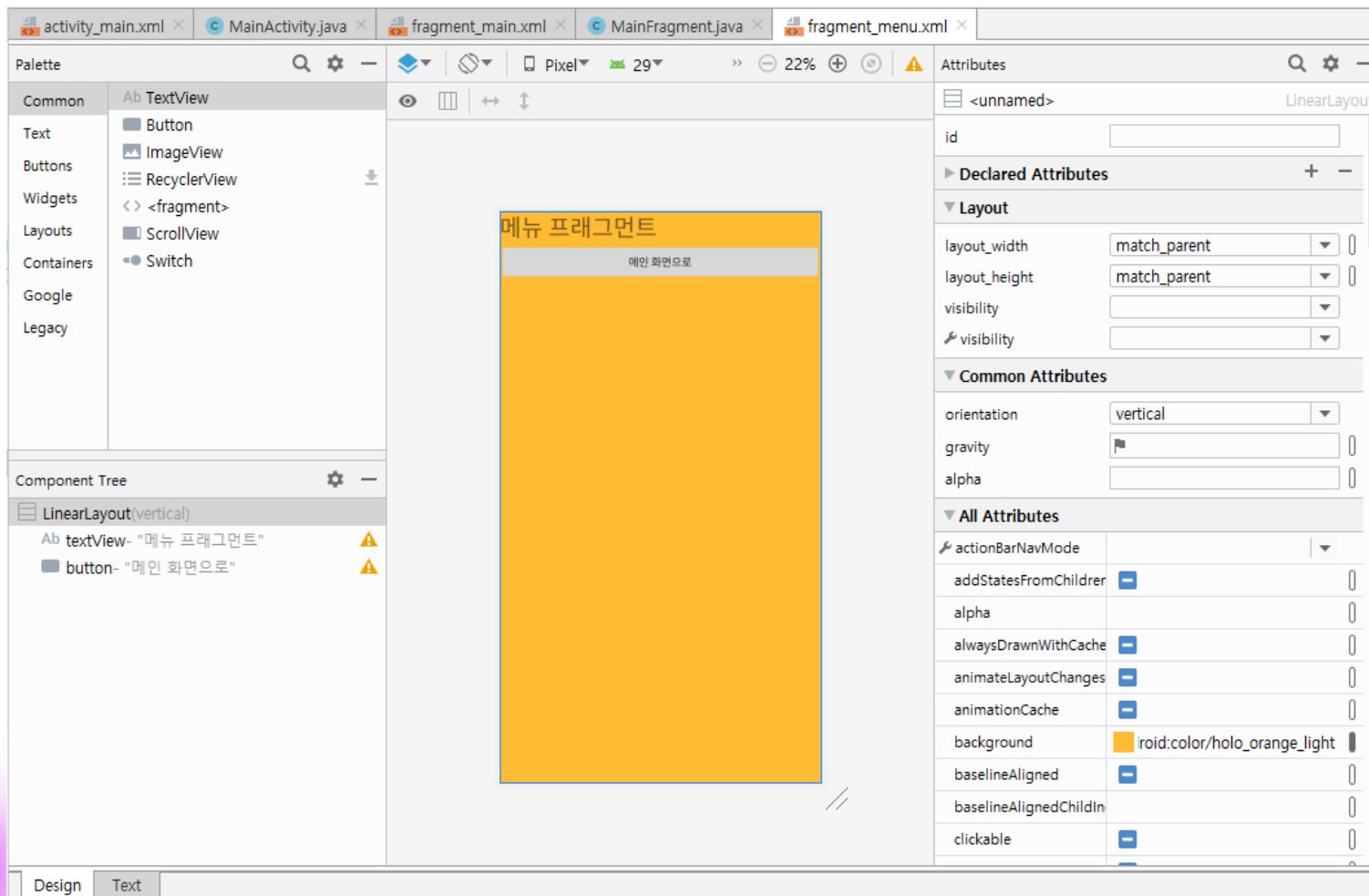
- 프래그먼트가 부분화면으로 추가됨





코드에서 프래그먼트 추가하기

- 메뉴 프래그먼트를 위한 XML 레이아웃 파일 만들고 화면 구성





코드에서 프래그먼트 추가하기

- 프래그먼트 소스 코드 입력

참조파일 SampleFragment>/app/java/org.techtown.fragment/MainFragment.java

```
public class MainActivity extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        ViewGroup rootView = (ViewGroup) inflater.inflate(R.layout.fragment_main,
                                                         container, false);

        Button button = rootView.findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                MainActivity activity = (MainActivity) getActivity();
                activity.onFragmentChanged(0);
            }
        });

        return rootView;
    }
}
```



코드에서 프래그먼트 추가하기

- 메인 액티비티 코드에서 프래그먼트 매니저 사용

참조파일 SampleFragment>/app/java/org.techtown.fragment/MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    MainFragment mainFragment;
    MenuFragment menuFragment;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

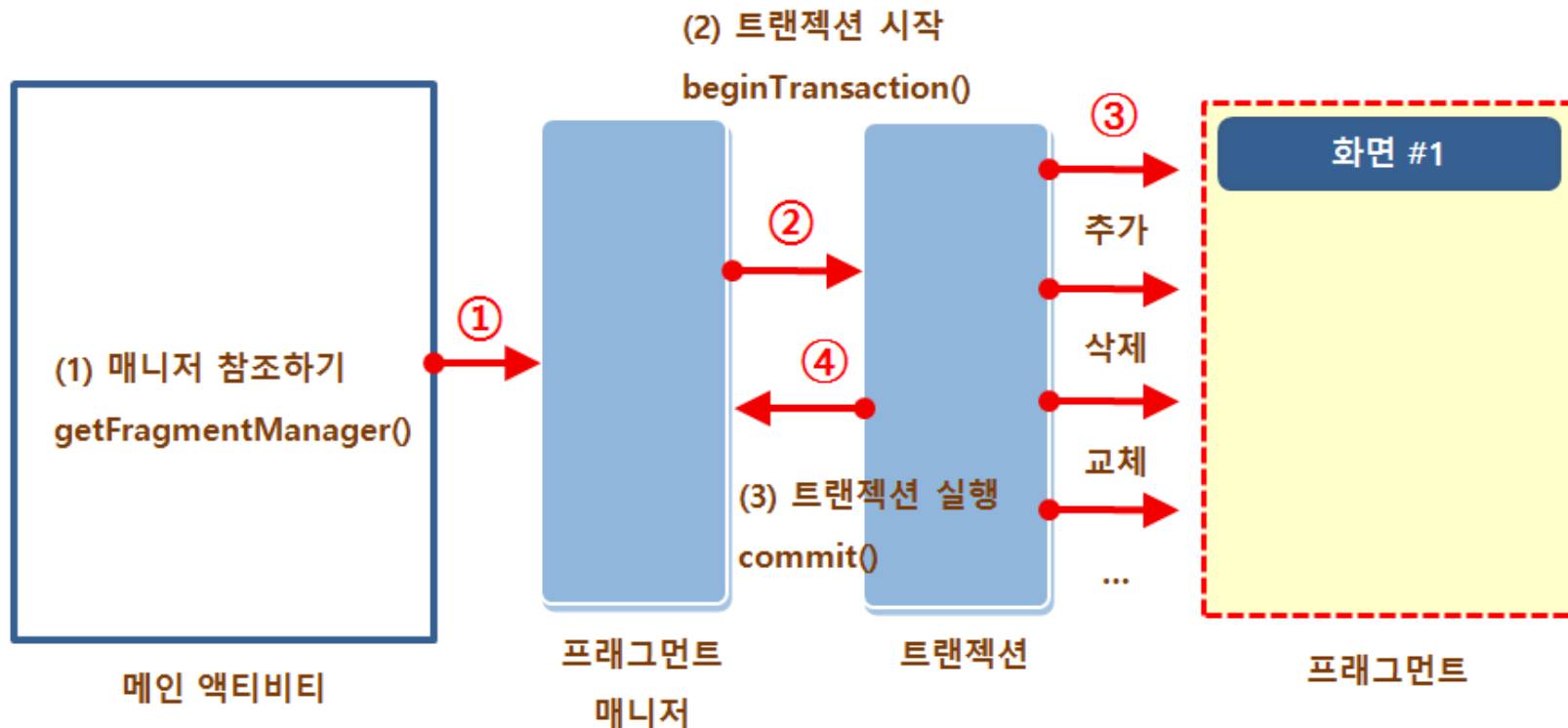
        mainFragment = (MainFragment) getSupportFragmentManager().findFragmentById(R.id.mainFragment);
        menuFragment = new MenuFragment();
    }

    public void onFragmentChanged(int index) {
        if (index == 0) {
            getSupportFragmentManager().beginTransaction().replace(R.id.container,
                                                                    menuFragment).commit();
        } else if (index == 1) {
            getSupportFragmentManager().beginTransaction().replace(R.id.container,
                                                                    mainFragment).commit();
        }
    }
}
```



코드에서 프래그먼트 추가하기

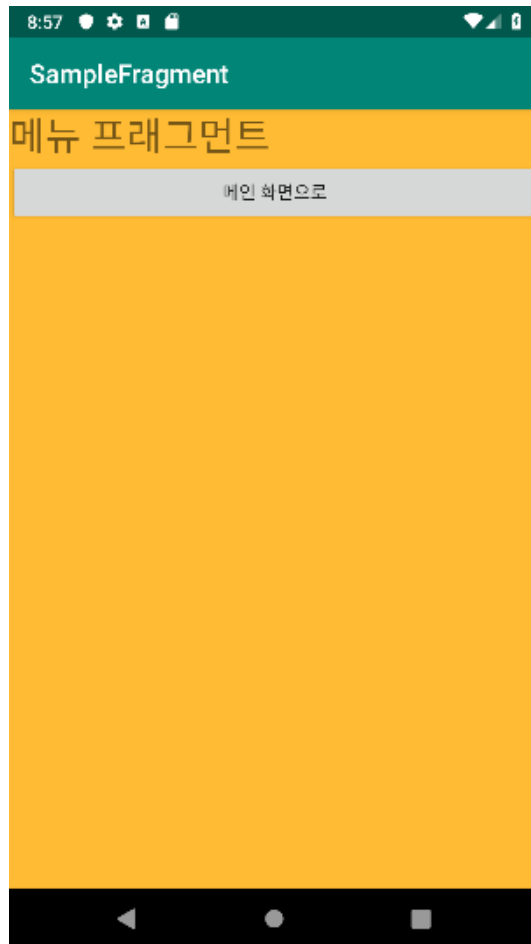
- 프래그먼트가 액티비티의 프래그먼트 매니저를 통해 의사소통하는 방식





앱 실행 결과

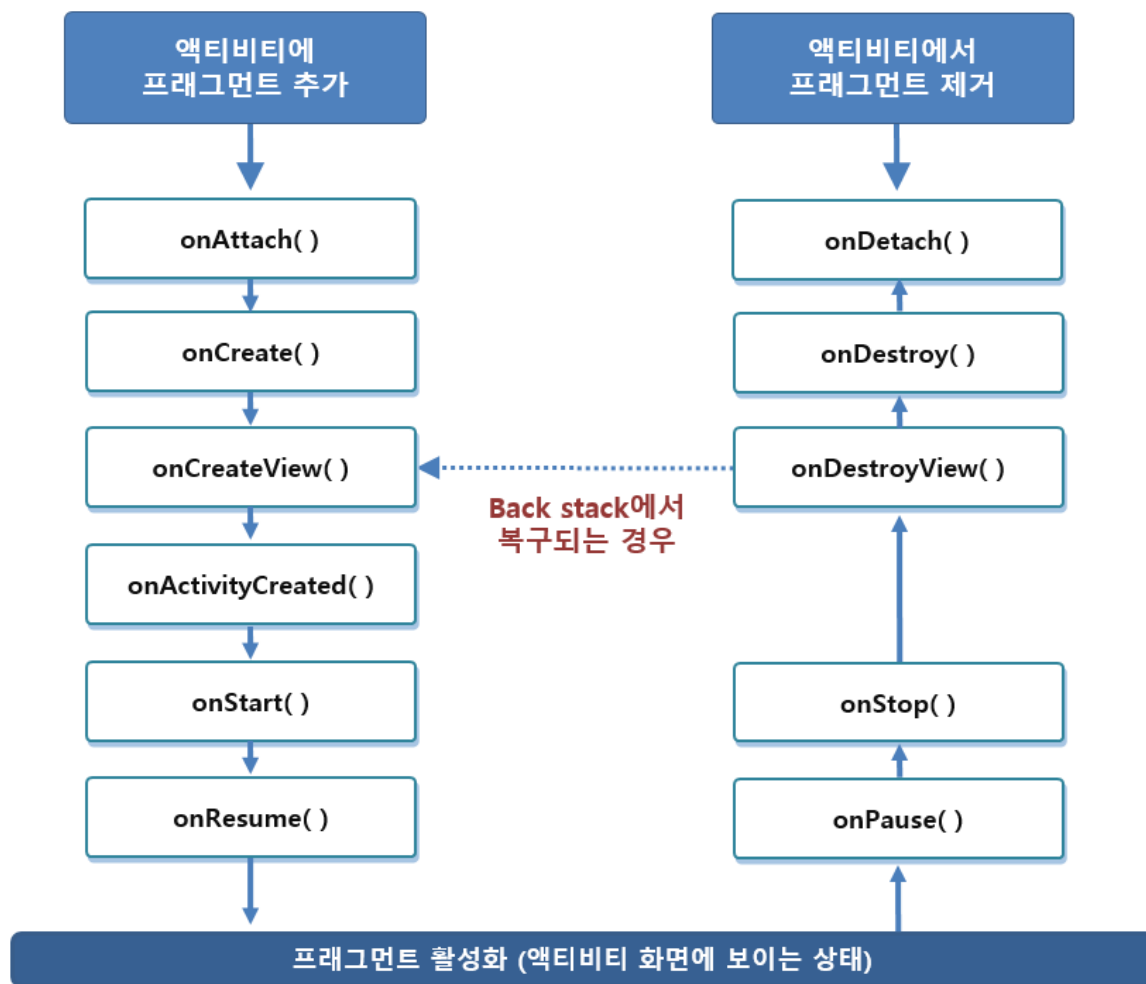
- 프래그먼트 안의 버튼 클릭 시 액티비티를 통해 프래그먼트 전환





프래그먼트의 수명주기

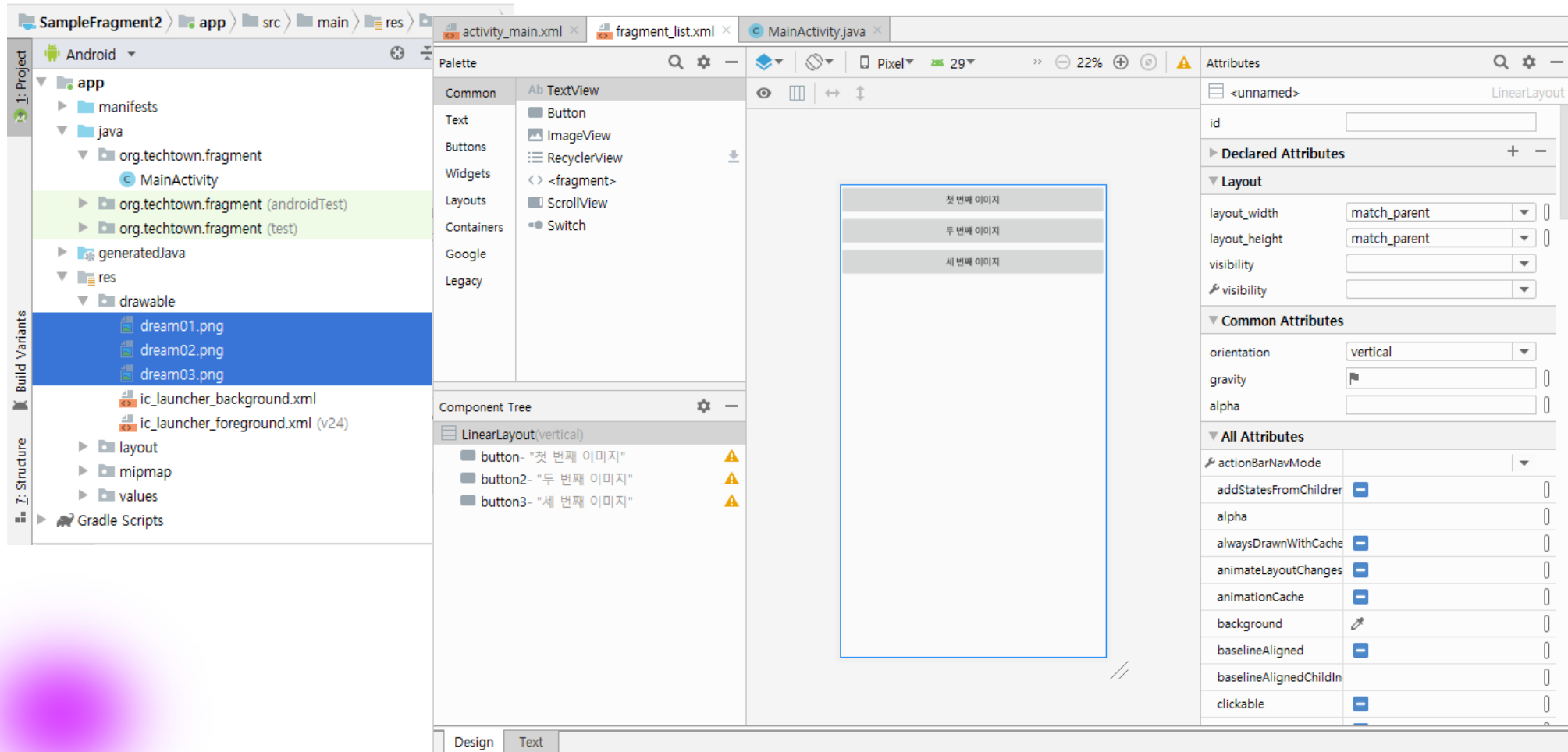
- 독립적인 상태 유지를 위해 수명주기 메소드를 제공함





두 개의 프래그먼트로 구성된 이미지 뷰어

- 이미지 복사한 후 프래그먼트 구성





두 개의 프래그먼트로 구성된 이미지 뷰어

- 첫번째 프래그먼트에 코드 추가

참조파일 SampleFragment2>/app/java/org.techtown.fragment/ListFragment.java

```
public class ListFragment extends Fragment {

    public static interface ImageSelectionCallback {
        public void onImageSelected(int position);
    }

    public ImageSelectionCallback callback;

    @Override
    public void onAttach(Context context) {
        super.onAttach(context);

        if (context instanceof ImageSelectionCallback) {
            callback = (ImageSelectionCallback) context;
        }
    }

    @Nullable
    @Override
```



두 개의 프래그먼트로 구성된 이미지 뷰어

- 첫번째 프래그먼트에 코드 추가

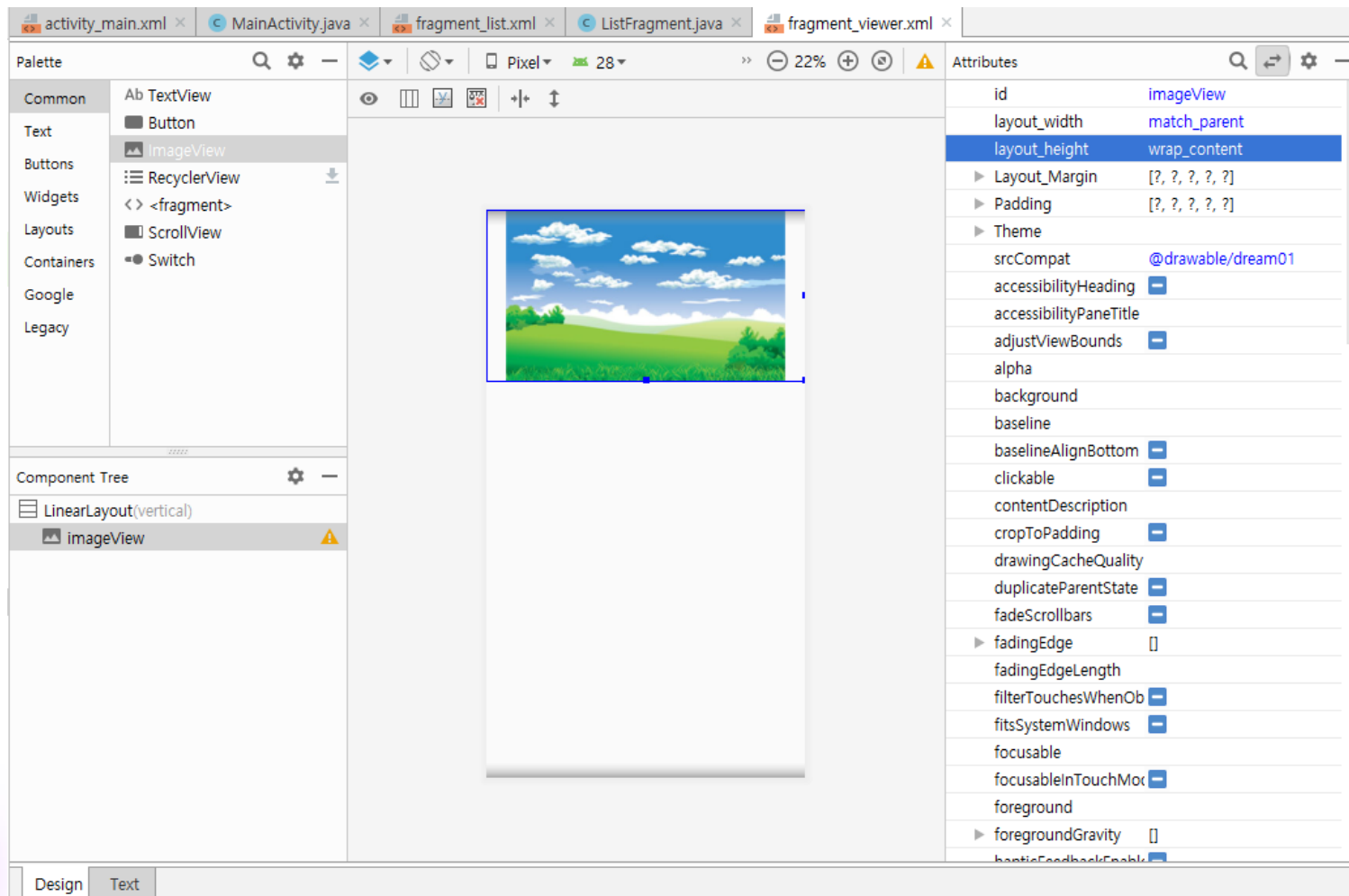
```
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
                        @Nullable Bundle savedInstanceState) {
    ViewGroup rootView = (ViewGroup) inflater.inflate(R.layout.fragment_list, container, false);
    Button button = rootView.findViewById(R.id.button);
    button.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            if (callback != null) {
                callback.onImageSelected(0);
            }
        }
    });

    Button button2 = rootView.findViewById(R.id.button2);
    button2.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            if (callback != null) {
                callback.onImageSelected(1);
            }
        }
    });
}
```



두 개의 프래그먼트로 구성된 이미지 뷰어

- 두번째 프래그먼트에 이미지뷰 추가





두 개의 프래그먼트로 구성된 이미지 뷰어

- 두번째 프래그먼트에 이미지뷰 추가

참조파일 SampleFragment2>/app/java/org.techtown.fragment/ViewerFragment.java

```
public class ViewerFragment extends Fragment {  
    ImageView imageView;  
  
    @Nullable  
    @Override  
    public View onCreateView(LayoutInflater inflater, @Nullable ViewGroup container,  
                             @Nullable Bundle savedInstanceState) {  
        ViewGroup rootView = (ViewGroup) inflater.inflate(R.layout.fragment_viewer,  
                                                         container, false);  
  
        imageView = rootView.findViewById(R.id.imageView);  
        return rootView;  
    }  
  
    public void setImage(int resId) {  
        imageView.setImageResource(resId);  
    }  
}
```



두 개의 프래그먼트로 구성된 이미지 뷰어

- 메인 액티비티의 XML 레이아웃에 두 개의 프래그먼트 추가

```
<fragment
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_weight="1"
    android:name="org.techtown.fragment.ListFragment"
    android:id="@+id/listFragment"
/>
```

```
<fragment
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_weight="1"
    android:name="org.techtown.fragment.ViewerFragment"
    android:id="@+id/viewerFragment"
/>
```



두 개의 프래그먼트로 구성된 이미지 뷰어

- 메인 액티비티 코드에서 버튼 클릭 시 이미지 설정

참조파일 SampleFragment2>/app/java/org.techtown.fragment/MainActivity.java

```
public class MainActivity extends AppCompatActivity
    implements ListFragment.ImageSelectionCallback {

    ListFragment listFragment;
    ViewerFragment viewerFragment;

    int[] images = {R.drawable.dream01, R.drawable.dream02, R.drawable.dream03};

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        FragmentManager manager = getSupportFragmentManager();
        listFragment = (ListFragment) manager.findFragmentById(R.id.listFragment);
        viewerFragment = (ViewerFragment) manager.findFragmentById(R.id.viewerFragment);
    }

    @Override
    public void onImageSelected(int position) {
        viewerFragment.setImage(images[position]);
    }
}
```



앱 실행 결과

